TokenWave

SWE-CoWork: Benchmarking Coding Agents as Digital Employees in Organizations

TokenWave AI

A software engineering agent working within an organization needs to establish what to build, implement and verify it, and report what it has delivered. We introduce SWE-CoWork, a benchmark of 101 development assignments across 101 real repositories, 11 languages, and 12 domains. Each organization contains 26 simulated colleagues with distinct responsibilities, bounded local knowledge, and controlled collaboration behavior, yielding 2,626 employee profiles across the benchmark. Requirements are distributed across accountable colleagues and project records, including boards, versioned documents, chat, reviews, and CI history. Each requirement is linked to a fixed fail-to-pass (F2P) and pass-to-pass (P2P) test contract, so it can be followed from clarification to delivery. A median task requires 2,955 production lines across 60 files and is evaluated by 32 executable tests. We evaluate eleven models on 101 tasks. Claude-Opus-5.5 and GPT-6 Astra pass 63.84% and 62.91% of F2P nodes, yet complete only 6.93% and 7.92% of tasks; the other nine models remain below 40% F2P. Supplying the complete specification raises F2P from 39.09% to 55.76% for Seed Evolving and from 62.91% to 78.84% for GPT-6 Astra, while task pass rates stay unchanged. Traces linked to individual requirements show that Claude-Opus-5.5 usually finds the right colleagues: it directs 93% of first questions to the accountable owner and obtains 64% of the requirements held by colleagues. It loses track of individual requirements before delivery. Deferred questions go unresolved, obtained requirements remain unsatisfied, and 31% of requirements reported as done fail their tests. SWE-CoWork identifies sustained requirement tracking, from clarification through implementation, verification, and reporting, as a central gap between capable coding agents and dependable digital software engineers.

Date: October 22, 2026
Website: https://tokenwave.ai/
Code: https://github.com/TokenWave-AI/swe-cowork-bench
Authors: Full author list in Appendix A

1 Introduction

Imagine a software organization in which a human leader sets the direction and agent programmers carry out the engineering work. The leader decides what to build, sets priorities, and resolves major trade-offs. Agents take over assignments, recover missing context, coordinate dependencies, implement changes, and verify delivery. One person could supervise many such engineers, reviewing outcomes while the agents manage everyday execution.

A *digital software engineer* needs to inherit work already in progress and carry it through implementation, verification, and handoff. A request might be as brief as "finish the 2026.09 upgrade; the previous owner is on leave." To complete it, the agent recovers earlier decisions, establishes which requirements are still open, and consults colleagues who hold relevant knowledge. These activities are part of the engineering assignment.

Engineering work within an organization. We introduce SWE-CoWork to evaluate agents working inside software organizations. Each organization contains *digital employees*: functional models of colleagues

with distinct responsibilities, bounded local knowledge, and controlled collaboration behavior. There are 26 colleagues per organization, giving 2,626 employee profiles across 101 organizations. The agent receives an ongoing assignment, establishes its requirements through the people and records of the organization, implements the changes, and verifies and reports the result.

Software engineers routinely inherit code and decisions produced by others. They spend substantial effort understanding existing systems, reconstructing past decisions, and finding people who can answer unresolved questions (Ko et al., 2007; LaToza et al., 2006; Sillito et al., 2008; Xia et al., 2018). Evidence is spread across code, documents, discussions, reviews, CI runs, and colleagues' knowledge, and the sources can disagree. A design document may describe an approach that has since been replaced. A successful CI run may refer to an earlier revision. A colleague may know a compatibility constraint without knowing which downstream component depends on it. A trace found during implementation may invalidate the initial diagnosis. Engineers check the provenance of this evidence, resolve conflicting accounts, and revise their plans as the work proceeds.

Existing coding benchmarks assess increasingly realistic changes in substantial repositories, including the SWE-bench lineage (Jimenez et al., 2024), DeepSWE (Huang et al., 2026), SWE Atlas (Raghavendra et al., 2026), FrontierSWE (Kondra et al., 2026), CursorBench (Cursor, 2026), and Real-SWE (Das et al., 2026). Studies of underspecification and clarification (Kim et al., 2026; Pu et al., 2026; Edwards and Schuster, 2026) examine whether agents can recover missing information, often by interacting with a single simulated user. These evaluations typically start with an engineering task that has already been selected for the agent. In SWE-CoWork, the agent joins work in progress, reconstructs its current state from an organization's records and colleagues, determines what remains to be done, and completes the assignment.

Each SWE-CoWork task combines a real open-source repository with an authored engineering workplace. Figure 1 illustrates the setting using a single Click requirement. In the issue description, the requirement is given directly to the agent. In SWE-CoWork, it forms part of an ongoing assignment. The agent recovers the relevant decisions, establish how the requirement relates to the current project state, consult its owner when necessary, implement and verify the change, and revise its interpretation when new evidence appears. Both versions have the same underlying requirement and executable tests.

Employees and distributed knowledge. A typical task distributes requirements across six accountable owners in a 26-person organization. A module maintainer, a compatibility liaison, and a release coordinator hold different parts of the engineering context. Their answers need to be checked against one another and against the repository. The agent has to identify whom to ask, provide enough context for the colleague to answer, follow referrals, and return to questions whose answers were deferred. Documents and chat preserve earlier decisions, including ones that have since been superseded. The agent determines which people and records contain the information needed for the current change.

Tasks and workplace. SWE-CoWork contains 101 development assignments drawn from 101 upstream repositories across 11 languages and 12 application domains. Each assignment combines a sequence of upstream changes into a cumulative development objective. A median task adds 2,955 production lines across 60 files and is graded by 32 executable verifier test nodes. The agent starts with a short onboarding note, a frozen repository, and access to a project board, versioned documents, chat, a staff directory, merge-request history, and CI records. Across the benchmark, these workplaces contain 40,836 chat messages. Requirements are distributed across colleagues, records may preserve outdated conclusions, CI evidence applies only to the revision that produced it, and relevant information may appear after implementation has begun (Figure 4). The agent decides which records to inspect, whom to consult, which evidence to trust, and how to implement and test the changes. Its final handoff document records the completed work, verification results, and remaining issues.

Requirements linked to owners and tests. Every scored requirement has an accountable owner, a designed route to discovery through workplace records, executable evidence, or that owner, and a link to the fail-to-pass (F2P) and pass-to-pass (P2P) test nodes that check it. The test contract is fixed and defined independently of the agent's trajectory. These links let us follow each requirement through the assignment: whether it was raised with its owner, disclosed, implemented, verified, and accurately reported in the final handoff.

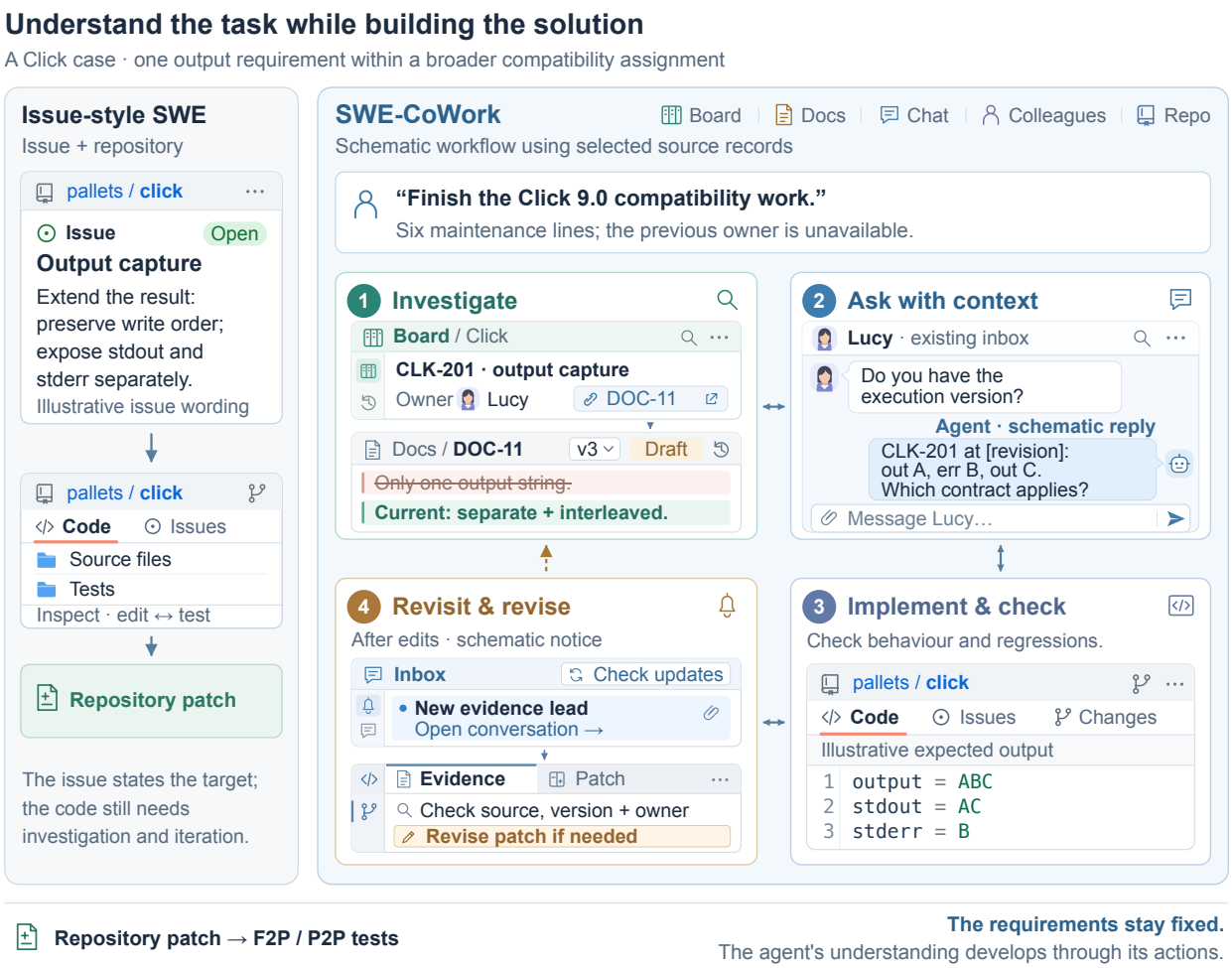


Figure 1 The same Click requirement presented directly as an issue-based task (left) and within an ongoing SWE-CoWork assignment (right). In SWE-CoWork, knowledge is distributed among digital employees and workplace records. The agent recovers the requirement, consults the responsible colleague, implements and verifies the change, and updates its interpretation when new evidence appears. Interfaces are schematic; Appendix E provides the source records.

Paired evaluation. For every SWE-CoWork task, we construct a paired Oracle-Spec condition that supplies the complete requirements at the start. The repository, tests, scaffold, and interaction budget are the same as in CoWork. Comparing performance under these conditions measures the additional difficulty of establishing requirements through the organization within a fixed execution budget. Runtime traces of searches, document reads, questions, referrals, CI inspection, code changes, and handoff notes then show where each requirement is lost: before it is raised with its owner, after deferral, after disclosure but before correct implementation, or between the delivered code and the reported status.

Results. We evaluate eleven models on 101 tasks and report F2P, P2P, and complete task pass rates (Table 2). Claude-Opus-5.5 and GPT-6 Astra pass 63.84% and 62.91% of F2P nodes but complete only 6.93% and 7.92% of tasks; the remaining nine models reach at most 39.09% F2P and 0.99% complete task success. Supplying the complete requirements raises pooled F2P from 39.09% to 55.76% for Seed Evolving and from 62.91% to 78.84% for GPT-6 Astra (Table 3). Task pass rates do not change: Seed Evolving completes one task in both conditions, and GPT-6 Astra stays at 7.92%.

Tracking each requirement to delivery. The strongest agent already finds the right people. Claude-Opus-5.5 directs 93% of first questions to the accountable owner, and only 3% of the requirements held by colleagues belong to owners it never contacts. It obtains 64% of these requirements. Its failures occur after contact,

when a requirement needs to stay open until it is resolved. Most undisclosed requirements belong to owners it has already contacted: the relevant card was never raised, or a deferred question was never revisited (153 of 450 deferred threads, with a median of 7.1 hours of budget remaining). Requirements that do reach the agent still account for 56% of failed linked tests. Final handoffs report 31% of "done" requirements that fail their tests. The agent gathers most of the context it needs and loses the state of individual requirements between clarification, implementation, verification, and reporting. Weaker models add an earlier gap: DeepSeek V4 Pro sends a median of 0 colleague messages per run.

Contributions.

- **A benchmark that links distributed requirements to executable delivery.** SWE-CoWork provides 101 development assignments across 101 repositories, 11 languages, and 12 domains. In each, requirements are held by colleagues with bounded responsibilities and by project records, and each requirement is tied to a fixed F2P/P2P test contract.
- **A paired evaluation of acquiring requirements through an organization.** The CoWork and Oracle-Spec conditions share the repository, tests, scaffold, and execution budget, isolating the cost of establishing requirements through colleagues and records. We report this comparison for Seed Evolving and GPT-6 Astra alongside CoWork results for eleven models.
- **An account of where each requirement is lost.** Linking colleague conversations, code changes, verification, and handoff notes to individual tests, we show that many models struggle to obtain the requirements from the organization. Frontier models such as Claude-Opus-5.5 reach the right owners, and even when a requirement reaches the agent, they lose track of it at a later stage: unraised cards, abandoned deferrals, unsatisfied disclosed requirements, early stops, and completion claims contradicted by tests.

2 Related Work

Engineering tasks in real repositories. Recent software engineering benchmarks cover a growing range of development activities in realistic repositories. DeepSWE (Huang et al., 2026; Datacurve, 2026) evaluates original tasks in real repositories; SWE Atlas (Raghavendra et al., 2026) covers codebase questions, test writing, refactoring, and implementation; FrontierSWE v2 (Kondra et al., 2026) examines implementation, optimization, and research tasks under long interaction budgets; CursorBench 4.0 (Cursor, 2026; Jain, 2026) derives tasks from real developer activity; and Real-SWE (Das et al., 2026) extends evaluation to private production codebases and business tooling. These benchmarks cover diverse engineering activities, larger development contexts, and longer execution horizons. Agents generally receive an identified development assignment and proceed to investigate and implement it. In SWE-CoWork, the agent enters an ongoing software project with a brief handoff. It establishes the current requirements from colleagues, project records, and repository history before completing and validating the implementation.

Clarification and requirement recovery. Several benchmarks examine how agents recover information missing from an initial request. RealSWE (Kim et al., 2026) varies the information provided for a fixed task, while LHAW (Pu et al., 2026) removes information along goal, constraint, input, and context dimensions. *Ask or Assume* (Edwards and Schuster, 2026) and Dialogue SWE-Bench (King and Flanigan, 2026) introduce interactive users who answer clarification questions, and ICAE-Bench (Peng et al., 2026) grounds user responses in withheld product constraints. τ^{τ}-Bench (Shi et al., 2026) requires developer agents to recover requirements from business artifacts and interactions with a client. These settings examine how information acquisition affects an agent's ability to understand and complete a development task.

SWE-CoWork extends requirement recovery to interactions with multiple employees who hold different parts of the relevant knowledge. Each digital employee has a defined responsibility, bounded local knowledge, and rules governing what it can disclose. The agent identifies the appropriate colleagues, follows up on their answers, and combines their information with project boards, documents, conversations, repository history, and execution results. Sources may reflect different revisions or points in time, creating disagreements that the agent resolves against the current project state. An answer may reveal another dependency, and information

Benchmark	Engineering task	Initial information	Information sources	Information process	Evaluation
DeepSWE 1.1 (Huang et al., 2026; Datacurve, 2026)	Repo change	Request Underspecified	Repo	n/r	Patch tests
SWE Atlas (Raghavendra et al., 2026)	Code Q&A Test writing Refactor	Request Underspecified	Repo	No live Q&A	Execution Rubric
FrontierSWE v2 (Kondra et al., 2026)	Build Optimize Research	Brief	Workspace Self-check	n/r	Task score
CursorBench 4.0 (Cursor, 2026; Jain, 2026)	Edit Refactor Investigate	Request Underspecified	Code Tools	n/r	Agentic grader
Real-SWE (Specific Labs) (Das et al., 2026)	Enterprise change	Brief Underspecified	Repo Docs Board Chat	n/r	n/r
RealSWE (paper v2) (Kim et al., 2026)	Bug fix Feature	Issue variants Underspecified	Repo	Static request No live Q&A	Fixed target
τ^{τ}-Bench (Shi et al., 2026)	Agent build	Underspecified	Docs APIs Code	Client Q&A Conflicting history Superseded records	User simulation
SWE-CoWork (ours)	Multi-PR Repo change	Onboarding brief Underspecified	Repo Board Docs Chat MR/CI	Multi-owner Q&A Versioned records Conflicting history Execution-time leads	F2P/P2P Fixed contract

task initial information information source interaction / dynamics evaluation

n/r = not reported in the cited public description.

Table 1 How recent software engineering benchmarks present engineering work to agents. *Initial information* describes what the agent receives at the outset; *information sources* identifies where evidence relevant to the task resides; and *information process* describes how agents clarify requirements, reconcile evidence, and respond to information encountered during execution. "n/r" indicates that the property is not described in the cited public material.

obtained during implementation may require revising an earlier decision. The agent therefore tracks the provenance and authority of evidence as it establishes requirements and carries out the work.

Agents in workplace environments. Workplace-agent benchmarks evaluate tasks that involve organizational records, applications, and interactions with colleagues. TheAgentCompany (Xu et al., 2025), for example, places agents in a simulated company with colleagues and workplace services to evaluate their ability to complete professional tasks. SWE-CoWork applies this organizational setting to substantial software engineering assignments. Its digital employees operate within a project environment connected to a real repository and its development history. Agents communicate with colleagues, inspect organizational records, and implement changes in the codebase. Final correctness is assessed through a fixed executable contract containing fail-to-pass and pass-to-pass tests.

The behavior of simulated colleagues also affects the reliability of interactive evaluation. Users simulated entirely by language models may reveal different information across runs (Seshadri et al., 2026). In SWE-CoWork, the environment specifies which facts each colleague can disclose and the conditions under which those facts become available. A language model expresses permitted responses in natural language. This design allows conversational variation while keeping disclosure rules and the underlying task contract fixed across runs.

Analyzing agent behavior and failures. Recent work also examines the trajectories through which coding agents complete tasks. SWE-RPG (Zhou et al., 2026) provides representations for studying planning and clarification behavior in software agent trajectories, while τ-bench (Yao et al., 2024) examines reliability across repeated interactive executions. Coding-agent frameworks such as SWE-agent, OpenHands, and Agentless (Yang et al., 2024; Wang et al., 2025; Xia et al., 2024) illustrate how agent scaffolding affects task execution and measured performance.

SWE-CoWork records the agent's searches, document reads, questions, referrals, execution evidence, code changes, and submitted patch. These records support analysis of how requirements are discovered, interpreted, and translated into code. A failed test may correspond to a requirement the agent never encountered, evidence it obtained but failed to incorporate, or an implementation that did not satisfy a correctly understood requirement. Examining these stages helps locate observable failures in the agent's workflow and compare behavior across models and scaffolds.

Table 1 compares recent benchmarks by their initial task information, available evidence sources, and information processes. Appendix B discusses additional repository, interactive, and workplace benchmarks.

3 Engineering Organizations of Digital Employees

Each SWE-CoWork task places a coding agent in an engineering organization consisting of a real software repository, a shared workplace, and a population of digital employees. The agent receives a brief handoff, establishes the requirements through workplace records and interactions with colleagues, and implements, verifies, and delivers the requested changes. Each task has a fixed executable contract evaluated by fail-to-pass (F2P) and pass-to-pass (P2P) tests. The design connects three elements for every scored requirement: the colleague or record that holds it, the responsibility boundary that determines who can answer for it, and the test nodes that check it. These links let us follow a requirement from the first question about it to the delivered code and the final handoff note.

3.1 Digital Employees

We construct an organization of 26 modeled colleagues around each repository, yielding 2,626 employee profiles across the 101 tasks. Each digital employee represents a named employee with a defined role, responsibilities, local knowledge, and rules governing interactions with the agent. Responsibilities and knowledge are grounded in the repository's files, declarations, and modules. The organization spans one target project and four neighboring projects that provide additional organizational context. Within this population, a median of six employees hold requirements that determine the task's executable outcome (Table 6).

What a digital employee represents. Each digital employee has three properties: responsibility, local knowledge, and collaboration behavior. *Responsibility* identifies the employee accountable for a module or decision. *Local knowledge* defines which requirements the employee can explain and which questions require referral to another colleague. *Collaboration behavior* governs how the employee responds. An owner may request a reproduction or caller path, address one topic at a time, or defer an answer until the agent follows up. The environment controls which information can be disclosed at each step, while a language model generates the employee's response.

From individual digital employees to an organization. The digital employees share project records that connect people to engineering work. Board cards identify owners, documents record decisions, and chat and review threads preserve earlier discussions. Information obtained from one employee may need to be checked against another employee's constraints or a more recent repository revision. The agent identifies the appropriate owners, resolves outstanding questions, and continues these interactions as implementation proceeds.

The agent interacts with the organization through a dedicated tool interface. It can search the staff directory, inspect the project board, open versioned documents, search and read chat, message colleagues, inspect merge requests and CI records, and check for new activity. Digital employees are accessible through the same workplace that contains their decisions and work history.

Workplace records are written in Chinese, reflecting the team that produced them, while repository code and executable contracts are primarily in English.

The handoff. The agent joins an engineering assignment already in progress. It begins with a short onboarding note that states the development objective and identifies workplace resources relevant to the current project state. The note specifies restrictions on modifying existing tests, excludes neighboring projects from the assignment, and identifies historical terminology that may differ from the current repository. The remaining acceptance criteria are distributed across workplace records and colleagues.

The agent decides which records to inspect, which references to follow, and which colleagues to contact. It interprets the available evidence against the current repository state and resolves questions that affect the implementation. At completion, it produces a delivery note describing the requirements established, the changes made, the validation performed, and any unresolved questions.

3.2 Composition

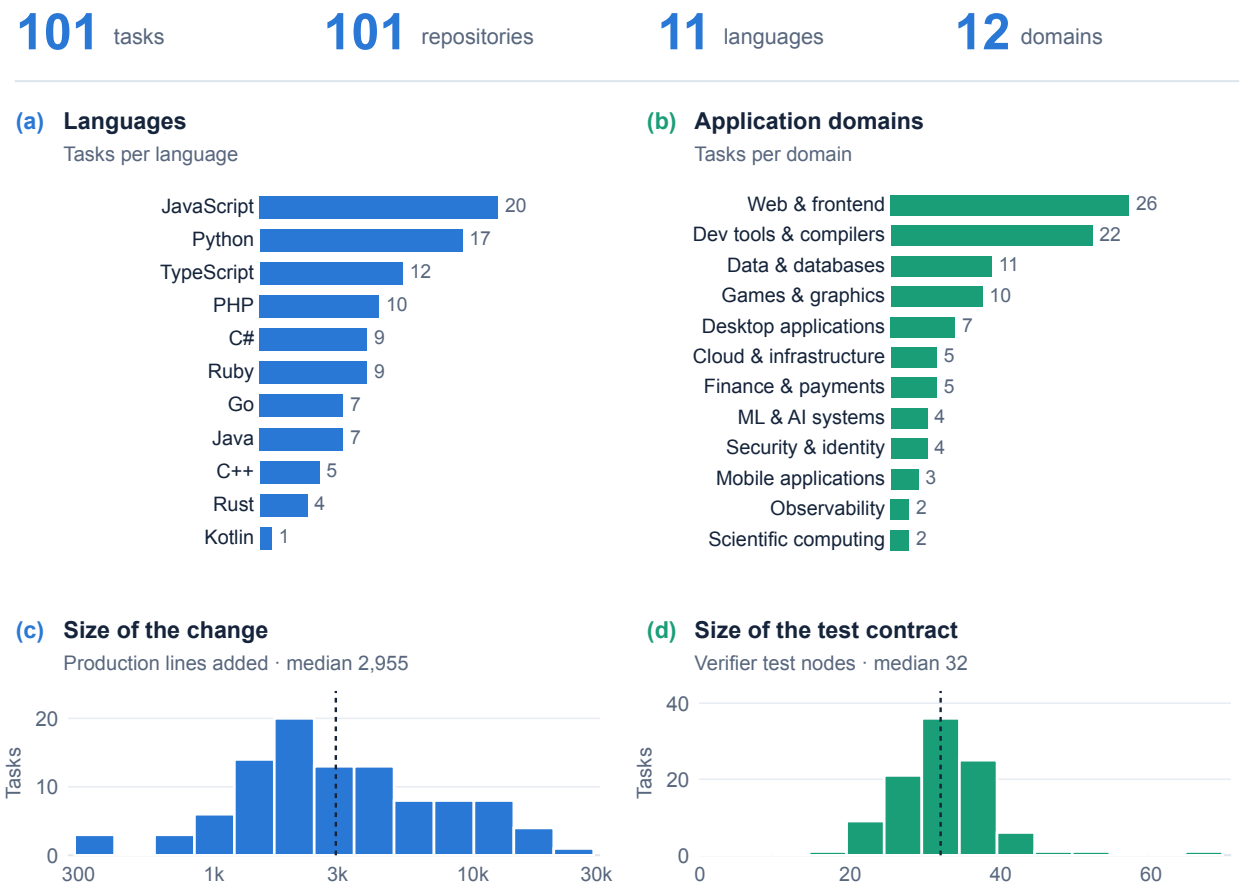


Figure 2 Composition of the 101 SWE-CoWork tasks. (a) Primary language of the production changes. (b) Application domain of the upstream repository. (c) Production lines added by the reference changes, shown on a logarithmic axis. (d) Verifier test nodes per task. Dashed lines mark medians.

SWE-CoWork contains 101 tasks drawn from 101 upstream repositories across 11 programming languages and 12 application domains, including web frameworks, compilers, games, payments, and scientific software (Figure 2; Appendix C). 11 tasks involve substantive changes in more than one programming language.

The reference solutions involve substantial engineering work. The median solution adds 2,955 lines of production code across 60 files, excluding tests, documentation, configuration, fixtures, and generated artifacts. For each task, we maintain a complete Oracle-Spec document that enumerates the requirements encoded

by the same executable contract used to evaluate the workplace assignment. Across the 101 tasks, these contracts contain 2,699 F2P nodes and 562 P2P nodes, giving 3,261 executable checks in total.

3.3 Shared Records and Organizational History

Each assignment is situated within records from a wider engineering organization. Across the benchmark release, these organizations contain 10,329 board cards, 5,097 versioned documents, 40,836 chat messages, 1,194 merge requests, and 2,626 colleague profiles. The initial onboarding note contains a median of 206 words, while the authored document corpus contains a median of 5,453 words. Requirements and supporting evidence are distributed across these records and their histories.

The agent determines which artifacts to inspect, which historical decisions still apply, which execution records correspond to the current revision, and which colleagues can provide missing information. We keep the overall organization size roughly comparable across tasks to limit variation in the volume of available workplace material.

Organizational history. The workplace records how projects develop over time. Versioned documents, board histories, review threads, CI records, and conversations contain intermediate diagnoses, superseded plans, rejected approaches, and observations tied to particular revisions. Historical records may use interface, module, or protocol names that differ from those in the current codebase. The agent establishes which claims remain applicable and reconciles them with subsequent decisions and the current repository.

Access follows the organization's information boundaries. Private groups and direct messages that do not include the agent are inaccessible. Records from neighboring projects provide organizational context, while those projects remain outside the scope of the assignment.

3.4 Scoring

We report three outcome metrics: **F2P**, the proportion of fail-to-pass verifier nodes satisfied by the submitted implementation; **P2P**, the proportion of pass-to-pass nodes preserved; and **Pass rate**, the proportion of tasks for which every F2P and P2P node passes in the same run. F2P and P2P are computed by pooling passed nodes and total nodes across tasks.

Each task contains multiple independently testable behaviors. F2P measures partial delivery of the required functionality, P2P measures preservation of existing functionality, and Pass rate measures completion of the full executable contract. Section 5 provides the aggregation formulas and describes the trajectory diagnostics.

4 Working a SWE-CoWork Task

In a SWE-CoWork task, the agent works with the digital employees introduced in Section 3.1. Starting from a brief handoff, it establishes the requirements through repository evidence, workplace records, and conversations with colleagues. The agent then implements and verifies the requested changes, revisiting earlier decisions as new information becomes available. We describe this workflow, examine how an owner's clarification changes an implementation in a recorded run, and present the information challenges represented in the workplace.

4.1 Working with the Digital Employees

Agents move between investigation, implementation, and clarification as they gather evidence. A typical run involves the following activities:

1. **Orient.** Read the onboarding note, inspect the repository, identify the target project, and establish the initial scope and relevant contacts.
2. **Investigate.** Search project boards, documents, chat, and engineering history for requirements and implementation constraints. Check record provenance, reconcile version differences, and trace references to renamed interfaces.

3. **Ask.** Identify the colleague responsible for an unresolved decision and ask a question supported by the available evidence. Follow referrals and respond to requests for additional context.

4. **Build and verify.** Implement the required changes, run relevant tests, and check for regressions in existing functionality and neighboring projects.

5. **Revise.** Examine new messages, traces, and corrections. Confirm their implications with the responsible colleague and update the implementation when necessary.

6. **Deliver.** Document the confirmed requirements, implementation changes, and verification results in the delivery note.

Every scored requirement has a route to discovery through workplace records or a colleague who holds the relevant knowledge. Several conversations may be active during implementation. One colleague may request a reproduction, another may refer the agent to a different owner, and a third may supply a constraint that requires a code change. The agent tracks the decisions established in these exchanges and their implications for the implementation.

Each digital employee discloses information according to its assigned knowledge, responsibilities, and conversation state. The environment controls which facts are available in each exchange, and the language model generates the responses. These rules maintain consistent knowledge boundaries across runs and support conversations in which the agent can ask follow-up questions and resolve outstanding decisions.

4.2 Inside a Run: An Owner Clarification Changes the Patch

Task 094 concerns the 2026.09 cumulative upgrade of Kimai, a PHP application for tracking time. The workplace contains 21 feature cards, 51 documents, and 26 digital employees, with the task owner temporarily away. In a run lasting three hours and 22 minutes, a frontier agent made approximately 1,000 tool calls, read 16 documents, reviewed 41 board cards, and sent 19 messages to six colleagues. One requirement illustrates how information from a digital employee changed the delivered implementation (Figure 3).

Card KIM-210 requires timesheet exports to express durations in decimal hours. The agent located the spreadsheet renderer, added an option for configuring the decimal base, and wrote a test using 3,600 seconds per unit. An entry lasting 5,400 seconds produced an exported value of 1.5 hours. Twelve focused tests passed with 74 assertions, confirming the numerical conversion.

The design document left the activation mechanism unspecified. The agent contacted the export owner with its implementation result and asked for confirmation:

> "My XLSX cell and total calculate to 1.5. Please confirm whether that is the intended decimal behavior…" — *agent*

> "…when the current user's `timesheet.export_decimal` preference is true, the value is 1.5 hours… The preference is evaluated on every export, even when one renderer is reused for different users." — *export owner*

The owner's response established two implementation requirements: the decimal format must follow the current user's `timesheet.export_decimal` preference, and the renderer must evaluate that preference on every export. The agent revised the renderer to read the current user's preference on each render and extended the tests to cover zero durations, durations exceeding 24 hours, totals, and consecutive exports for two users sharing one renderer.

The clarification affected how the feature was activated and how the renderer handled user state. Reading the preference on every render ensures that one user's formatting choice does not carry over to another user's export. Across the full task, the agent delivered 17 of 21 new behaviors and preserved all 15 existing behaviors.

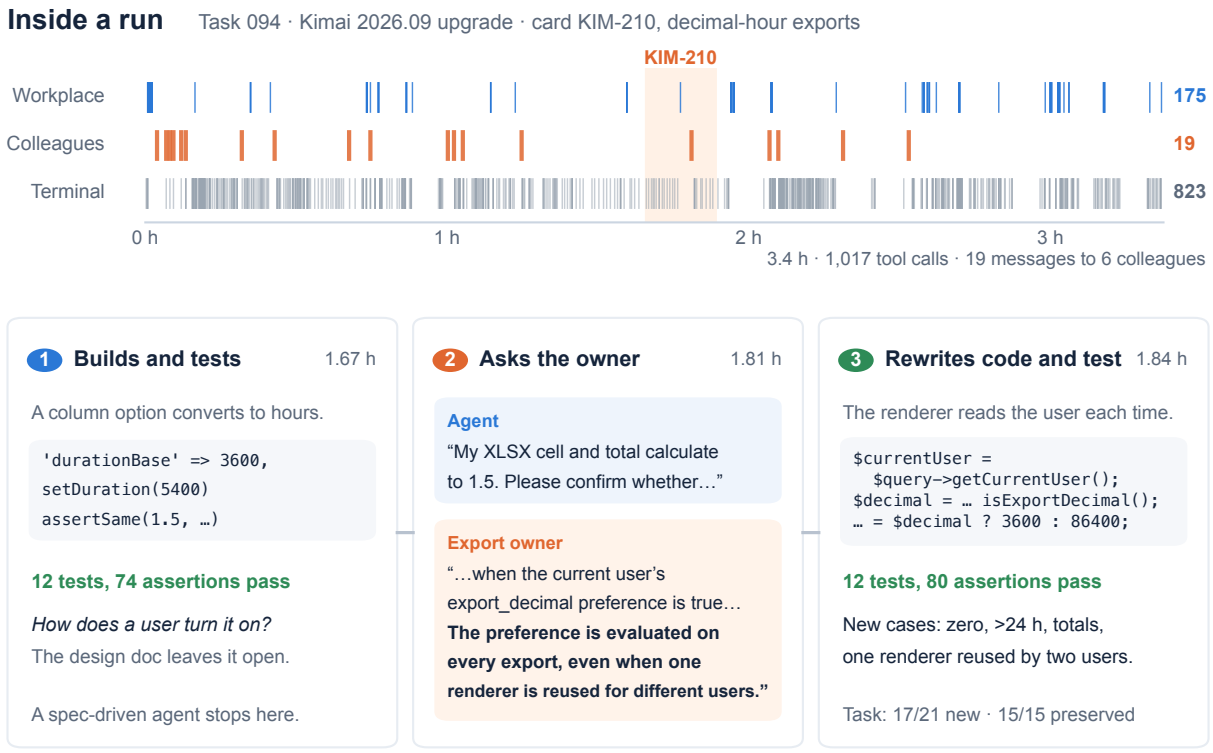


Figure 3 An owner clarification changes a timesheet export implementation in a recorded SWE-CoWork run (task 094, Kimai). The agent initially implements and tests decimal conversion, then contacts the export owner to resolve how users activate the feature. The owner specifies that the current user's preference must be evaluated on every render. The agent updates the implementation and expands the tests accordingly. Times indicate elapsed hours from the start of the run, and quotations come from the recorded exchange. This example comes from a pilot run on an earlier package revision and is separate from the evaluation cohort in Section 6.1.

4.3 Workplace Information Challenges

Engineering requirements are distributed across project boards, documents, engineering records, and colleagues. Establishing what a record means requires checking when it became available, where it came from, and who has authority over the decision. Figure 4 groups these information challenges into five situations and shows the engineering judgment required in each. They affect how the agent establishes the requirements of a fixed executable contract. Each misleading record has an accessible correction or a responsible colleague who can establish which evidence applies.

Late evidence. A recovered trace, corrected version label, or message from another team may arrive after implementation begins. In the schematic example, a successful browser record is followed by an SDK observation and a request to confirm the client version. The agent must follow this lead to the responsible owner, establish which behavior it concerns, and reassess the affected code and tests. Owners remain available for consultation before a scheduled notice appears.

Superseded conclusions. An earlier proposal can remain searchable after a published correction. For Stencil's `AUR-202`, the draft of `DOC-13` proposes an empty object when no source map is generated. The published third version instead requires the returned `map` field to remain `undefined`, while retaining the chunk's other output information. The agent must compare revisions and publication states to identify the applicable rule.

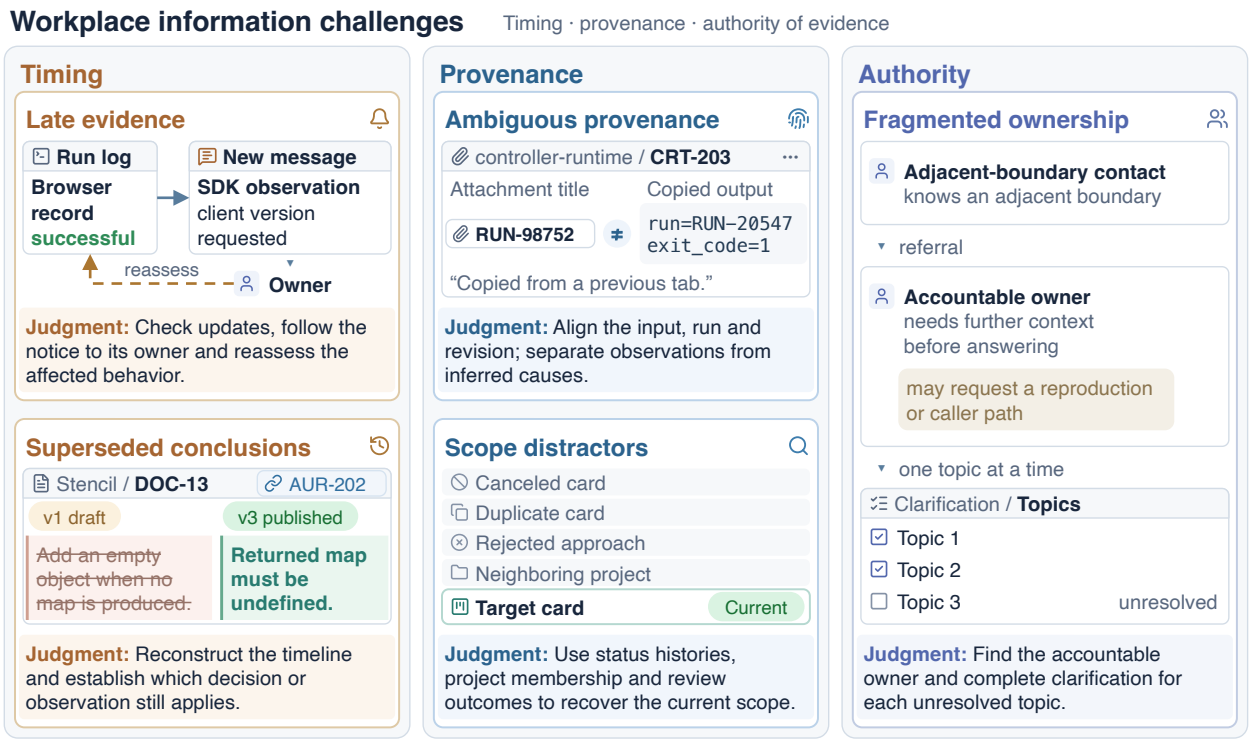


Figure 4 Five workplace information challenges and the judgments they require, organized by when evidence becomes available or is superseded, which run or project it belongs to, and who can resolve the outstanding decision. The panels use schematic situations and condensed workplace-record examples. Source details for Stencil and controller-runtime appear in Appendix E.

Ambiguous provenance. A log's title, contents, and surrounding discussion may refer to different executions. In the controller-runtime example, the attachment title names `RUN-98752`, but the copied output identifies `RUN-20547` and states that it came from a previous tab. The agent must align the output with its input, run, and code revision before using it to diagnose a failure. An observed error alone does not establish its cause or scope.

Scope distractors. Canceled cards, duplicates, rejected approaches, and neighboring projects remain visible alongside the current assignment. Similar wording can therefore lead to a record outside the required change. Status histories, project membership, and review outcomes establish which card is current and which approaches have already been rejected. The agent uses these distinctions to recover the target scope.

Fragmented ownership. A colleague may know an adjacent interface boundary yet lack authority over the unresolved decision. A referral leads to the accountable owner, who may request a reproduction, caller path, or version before answering. Clarification also proceeds topic by topic: resolving one question can leave others open. The agent supplies the requested context and tracks which decisions each exchange establishes before carrying them into the implementation.

These situations make investigation and coding interdependent. A newly located record can prompt another question, whose answer changes the patch or its tests. Appendix E gives the underlying records for the Stencil and controller-runtime examples and a further Click example involving output semantics and stream ownership.

5 Evaluation

We evaluate coding agents working within organizations of digital employees. Eleven models are evaluated in the CoWork setting using executable outcomes and collaboration records. A paired comparison with

ORACLE-SPEC measures the performance of Seed Evolving and GPT-6 Astra when complete requirements are supplied at the start. We also analyze Claude-Opus-5.5 trajectories to examine how agents obtain requirements, incorporate them into code, verify their implementations, and report their work.

5.1 Primary metrics

We report **F2P**, **P2P**, and **Pass rate** on $N = 101$ tasks. For task i, let f_i and p_i denote the numbers of passing F2P and P2P nodes, and let F_i and P_i denote their respective totals. All metrics are expressed as percentages:

$$\text{F2P} = 100\,\frac{\sum_{i=1}^{N} f_i}{\sum_{i=1}^{N} F_i}, \qquad \text{P2P} = 100\,\frac{\sum_{i=1}^{N} p_i}{\sum_{i=1}^{N} P_i},$$

$$\text{Pass rate} = \frac{100}{N}\sum_{i=1}^{N} \mathbf{1}[f_i = F_i \ \wedge \ p_i = P_i]\,.$$

F2P measures how many new requirements are satisfied, while P2P measures whether existing behavior is preserved. Both metrics aggregate test nodes across tasks, giving greater weight to tasks with more nodes. Pass rate measures the fraction of tasks for which every F2P and P2P node passes.

All models are evaluated against the full release contract: 2,699 F2P nodes, 562 P2P nodes, and 101 tasks. Only passes confirmed by the verifier count toward the numerators. For trajectory analysis, we use the fraction of F2P nodes passed within each task to relate outcomes to agent behavior.

5.2 Experimental design

CoWork evaluation. We evaluate eleven models at max effort in the COWORK setting: Claude-Opus-5.5 (Anthropic, 2026), GPT-6 Astra (OpenAI, 2026), Seed 2.1 Pro (ByteDance Seed, 2026a), ByteDance's Seed Evolving (ByteDance Seed, 2026b), Kimi K3 (Moonshot AI, 2026), DeepSeek V4 Pro (DeepSeek AI, 2026b), DeepSeek V4.1 Flash (DeepSeek AI, 2026a), GLM 5.3 (Z.ai, 2026b), GLM 5.2 (Z.ai, 2026a), and Qwen3.8-Max and Qwen3.8-Flash (Alibaba Cloud Model Studio, 2026).

GPT-6 Astra uses a Codex scaffold, and the other ten models use Claude Code scaffolds. Each agent starts with a brief handoff and access to the organization. It identifies the requirements through workplace records and conversations with colleagues, implements the requested changes, and submits its work for F2P/P2P evaluation. Table 2 reports F2P, P2P, and Pass rate for all eleven models on the 101-task release contract.

Oracle-Spec comparison. We evaluate Seed Evolving and GPT-6 Astra under two conditions. In COWORK, the agent obtains requirements through workplace records and colleagues. In ORACLE-SPEC, the complete requirement set is provided at the start. Both conditions cover the same 101 tasks, with identical task definitions, repository states, and executable test contracts.

We compare F2P, P2P, and Pass rate across the paired runs to measure how access to complete requirements affects implementation outcomes.

Claude-Opus-5.5: trajectory analysis. We examine Claude-Opus-5.5's COWORK trajectories to trace requirements from workplace evidence to implementation. For each run, we identify which requirements reached the agent, which remained unresolved, how the agent followed up with colleagues, and how it verified and reported its work.

We select Claude-Opus-5.5 because it achieves the highest CoWork F2P score and has the most complete trajectory archive among the evaluated models. The analysis covers its recorded runs and examines the relationship between requirement acquisition, agent behavior, and implementation outcomes.

5.3 Supplementary analysis metrics

Model calls and token usage. For nine models with usage summaries, we report the median per-run number of model calls, input tokens, output tokens, tool calls, and context compactions. Input usage accumulates

tokens processed across calls within a run, including repeated context. We pair these usage medians with each model's F2P on the full release contract. The usage snapshots cover different run cohorts, so this is a descriptive comparison across models. Figures 6–8 report these comparisons, with missing medians omitted. The available snapshots do not provide token medians for Seed 2.1 Pro or DeepSeek V4 Pro; these models remain in the eleven-model outcome and acquisition comparisons.

Requirement acquisition and collaboration. Each task's ORACLE-SPEC enumerates its requirements and identifies the responsible colleague or public record. Disclosure logs and saved tool replies establish which requirements were released by colleagues and which replies reached the agent.

We measure **ask rate**, the fraction of requirements held by colleagues that the agent raised with the responsible owner or obtained; **targeting**, the fraction of queried cards whose first question was directed to the accountable owner; **gain per question**, the number of disclosures per colleague message; **follow-through**, the fraction of raised requirements that were subsequently disclosed; **redundancy**, questions about cards whose requirements had already been fully disclosed; and **unread disclosures**, replies containing disclosed requirements that were absent from the agent's recorded observations.

6 Results

We evaluate how agents carry an assignment from an initial handoff to verified delivery. We first compare eleven models on the executable task contract, then measure the effect of supplying complete requirements to Seed Evolving and GPT-6 Astra. We then follow individual requirements through Claude-Opus-5.5 trajectories, from clarification through implementation, verification and the final handoff note.

6.1 Results Across Models

Table 2 compares eleven models on the 101-task CoWork benchmark. F2P measures delivery of new behaviors, P2P measures preservation of existing behaviors, and Pass rate requires every F2P and P2P node to pass in the same run.

Model	F2P (%)	P2P (%)	Pass rate (%)
Claude-Opus-5.5	**63.84**	93.24	6.93
GPT-6 Astra	62.91	89.15	**7.92**
Seed Evolving	39.09	94.48	0.99
Qwen3.8-Max	35.70	91.80	0.00
GLM 5.3	31.27	87.72	0.99
Qwen3.8-Flash	30.70	93.40	0.00
DeepSeek V4.1 Flash	28.20	94.30	0.00
Kimi K3	27.08	84.88	0.99
GLM 5.2	24.40	**95.20**	0.00
Seed 2.1 Pro	21.01	81.85	0.00
DeepSeek V4 Pro	13.04	90.21	0.00

Table 2 CoWork evaluation on 101 tasks. F2P and P2P use the release totals of 2,699 and 562 nodes, respectively. Pass rate requires every F2P and P2P node to pass in the same run. All scores are percentages, with higher values indicating better outcomes. Shading marks the top three per metric, from darkest (best, bold) to lightest; ties share a shade. Rows are ordered by F2P.

Partial delivery and complete task success. Claude-Opus-5.5 and GPT-6 Astra achieve the highest observed F2P scores, 63.84% and 62.91%, respectively. Their complete task Pass rates are 6.93% and 7.92%. Each of the remaining nine models has a Pass rate of at most 0.99%. Seed Evolving ranks third on F2P (39.09%) and completes one task. The F2P scores fall into two tiers: the two leading models exceed 62%, whereas the

other nine range from 13.04% to 39.09%, led by Seed Evolving and Qwen3.8-Max (35.70%). Even the leading models satisfy the full executable contract in fewer than one in ten runs.

P2P scores range from 81.85% to 95.20%, with different rankings from F2P. GLM 5.2 achieves the highest P2P score while satisfying 24.40% of F2P nodes. DeepSeek V4 Pro preserves 90.21% of P2P behaviors and satisfies 13.04% of F2P nodes. The results show considerable variation in the delivery of new behaviors, consistently stronger preservation of existing behaviors, and low rates of complete task success.

Requirement acquisition follows delivery. Figure 5 plots each model's F2P score against the share of requirements it obtained from colleagues and against its median number of tool calls per run. Table 9 in Appendix H lists all values, including colleague messages and wall-clock hours. The colleague rate counts the requirements held only by colleagues that a colleague disclosed during the run, as recorded by the disclosure events of the colleague service.

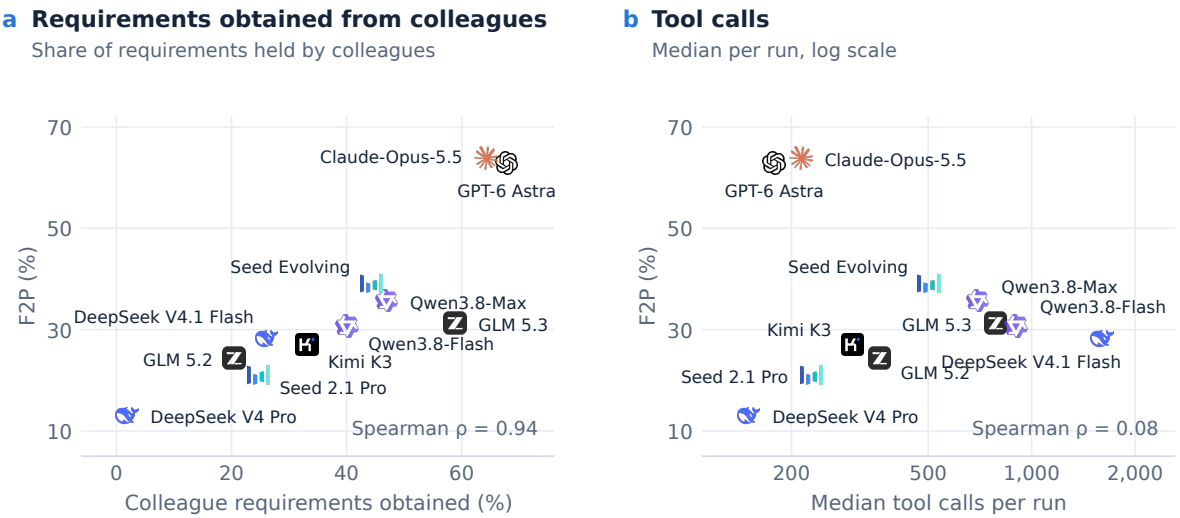


Figure 5 Requirement acquisition, tool calls, and delivery across eleven models. (a) F2P against the share of requirements held only by colleagues that a colleague disclosed during the run. (b) F2P against the median number of tool calls per run, on a logarithmic axis. Each model is shown by its developer's logo. Spearman ρ is computed over the eleven models.

The colleague rate orders the models almost as F2P does (Spearman $\rho = 0.94$ over the eleven models). GPT-6 Astra and Claude-Opus-5.5 obtain 67.8% and 64.2% of the requirements held by colleagues and lead on F2P. DeepSeek V4 Pro sends a median of 0 messages, obtains 1.8% of the requirements held by colleagues, and has the lowest F2P score, 13.04%. Its median run lasts 0.9 hours with 150 tool calls, and the agent works mostly from the handoff and the repository.

Effort follows a different order. Tool calls show almost no rank correlation with F2P ($\rho = 0.08$). DeepSeek V4.1 Flash makes a median of 1,603 tool calls and Qwen3.8-Max 696, and the median run of both reaches the 8-hour budget. They obtain 26.1% and 46.9% of the requirements held by colleagues, with medians of 6 and 10 messages. GPT-6 Astra obtains 67.8% with 178 tool calls in a median of 3.0 hours. The additional time and tool calls go into the repository and seldom into asking colleagues.

GLM 5.3 shows that obtaining the requirements is not enough. It sends the most messages, a median of 23, and obtains 58.8% of the requirements held by colleagues, the third highest rate. Its F2P score of 31.27% is about half that of the two leading models, whose colleague rates are only 5 to 9 points higher. GLM 5.3 receives most of the requirements and turns far fewer of them into passing tests. The paired Oracle-Spec experiment below examines how delivery changes when every requirement is provided directly, and Section 6.3 follows obtained requirements through implementation and verification.

Model calls and token use vary beyond delivery scores. Figure 6 compares the available usage medians with full-release F2P for nine models. Claude-Opus-5.5 uses a median of 174 model calls, 47.57 million input tokens, and 0.210 million output tokens, alongside 63.84% F2P. GPT-6 Astra uses a median of 149 model calls and 20.32 million input tokens per run, alongside 62.91% F2P. DeepSeek V4.1 Flash uses 981 calls and 42.49 million input tokens, with 28.20% F2P. The difference is larger for generated tokens: its median output is 2.410 million tokens, compared with 0.098 million for GPT-6 Astra. Kimi K3 has the lowest median input usage in this snapshot, 18.45 million tokens, with 27.08% F2P. Thus, neither a larger token volume nor a smaller input footprint consistently identifies the model with higher delivery scores. The usage cohorts differ, so these comparisons describe the observed model summaries rather than a matched-task efficiency estimate.

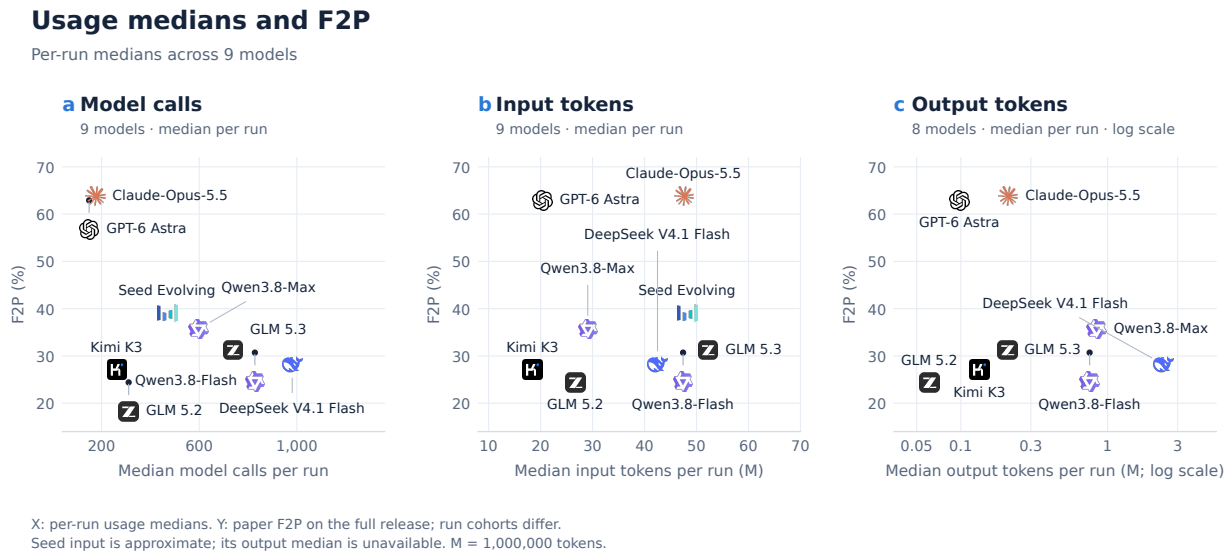


Figure 6 Model calls, token usage, and delivery. F2P on the full release against per-run medians of (a) model calls, (b) input tokens, and (c) output tokens. Each point is a model; token axes use millions (M), with a logarithmic scale in (c). Dots and leaders mark exact values where nearby logos are offset. Seed Evolving's input median is approximate; its output median is unavailable. Usage cohorts differ across models.

Tool use and context compaction. Figure 7 shows a similar spread in tool use and context compaction. Claude-Opus-5.5 has medians of 199 tool calls and 0 context compactions; GPT-6 Astra has 182 and 1, respectively. DeepSeek V4.1 Flash has 1,603 tool calls and 33 compactions, while Qwen3.8-Max has 696 and 11. Higher tool activity and more frequent compaction therefore coexist with lower delivery scores in this comparison.

Additional usage comparison. Figure 8 compares five models on calls and token usage. GPT-6 Astra has a median of 152 model calls and 0.102 million output tokens; DeepSeek V4.1 Flash has 1,060 calls and 2.546 million output tokens. Claude-Opus-5.5 combines the highest F2P with 174 calls and 0.210 million output tokens. Its median input usage, 47.57 million tokens, lies between Qwen3.8-Max (43.66 million) and DeepSeek V4.1 Flash (69.46 million).

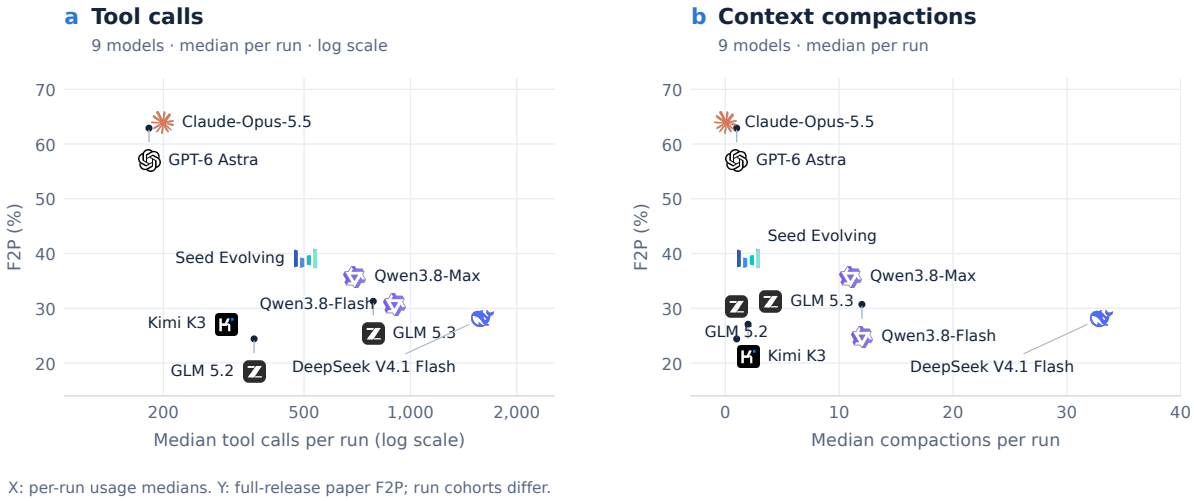


Figure 7 **Tool use, context compaction, and delivery.** Full-release F2P against per-run medians of (a) tool calls and (b) context compactions for nine models. Tool calls use a logarithmic axis; compactions use a linear axis to include zero. Dots and leaders mark exact values where nearby logos are offset.

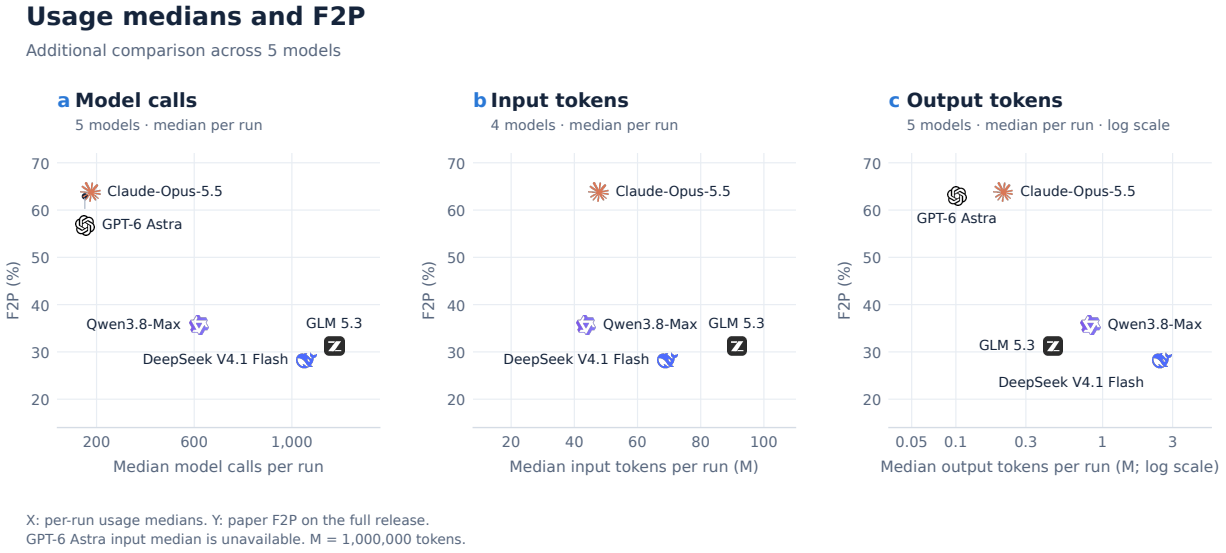


Figure 8 **Additional comparison of usage medians and delivery.** Full-release F2P against median (a) model calls, (b) input tokens, and (c) output tokens for five models. Input usage is shown for the four models with a reported median. Token axes use millions (M); (c) uses a logarithmic scale. Dots and leaders mark exact values where nearby logos are offset.

6.2 Effect of Supplying Complete Requirements

The paired Oracle-Spec experiment evaluates Seed Evolving and GPT-6 Astra on the same 101 tasks with complete requirements supplied up front. Both conditions use the same repositories, executable contracts, scaffold and execution budget. The comparison measures the effect of supplying a complete specification within a fixed execution budget.

	Seed Evolving			GPT-6 Astra		
Metric	CoWork	Oracle	Gain	CoWork	Oracle	Gain
F2P	39.09	55.76	+16.67	62.91	78.84	+15.93
P2P	94.48	89.50	−4.98	89.15	91.81	+2.66
Pass rate	0.99	0.99	+0.00	7.92	7.92	+0.00

Table 3 CoWork vs. Oracle-Spec for Seed Evolving and GPT-6 Astra. Scores are percentages; gain is in percentage points. Pooled metrics use the complete 101-task release contract.

Providing the specification raises Seed Evolving's F2P from 39.09% to 55.76%, a gain of 16.67 percentage points (Table 3). GPT-6 Astra's F2P rises by a similar amount, from 62.91% to 78.84%, a gain of 15.93 points. P2P moves in opposite directions: Seed Evolving's falls from 94.48% to 89.50%, while GPT-6 Astra's rises from 89.15% to 91.81%. With the complete specification, Seed Evolving implements more of the new behavior and preserves less of the existing behavior.

The observed gain reflects both direct access to requirements and the effort saved in searching for and confirming them. The experiment measures their combined contribution. Complete task success does not improve. Both models complete the same share of tasks in the two conditions: 0.99% for Seed Evolving and 7.92% for GPT-6 Astra. With every requirement in hand, Seed Evolving still leaves 44% of F2P nodes unsatisfied and GPT-6 Astra 21%. Establishing requirements through the organization accounts for about 16 F2P points for both models. The remaining failures occur after the requirements are available, in implementation and verification. The Claude-Opus-5.5 analysis below follows individual requirements through both stages.

6.3 Claude-Opus-5.5: Following Each Requirement to Delivery

We analyze 101 CoWork trajectories from Claude-Opus-5.5, which achieves the highest observed F2P score and has the most complete trajectory archive. We link colleague questions and replies, code changes, verification actions and delivery notes to the requirements covered by the final verifier. Each requirement then has a recorded path through four stages: clarification with its owner, follow-up on deferred answers, implementation and verification, and the status reported at handoff. The agent usually reaches the right owner. The subsections below show where individual requirements are lost after that contact. The results describe this model and the recorded cohort.

6.3.1 Clarification: reaching owners and raising requirements

Most undisclosed requirements belong to owners already contacted. The mean first-question owner targeting rate across runs is 93% (Table 7), indicating that the agent usually directs its initial question to the accountable owner. Of the 2,176 requirements held by colleagues, 64% are obtained (Figure 9a). For 16%, the agent raises the relevant card with its owner without obtaining disclosure. For another 17%, it contacts the owner about other cards without raising the relevant one. The remaining 3% belong to owners who are never contacted. Most undisclosed requirements are therefore associated with owners whom the agent has already contacted. The acquisition shortfall is concentrated in incomplete coverage of their requirements and conversations that end without disclosure.

Disclosure is associated with higher F2P. F2P nodes linked to disclosed requirements held by colleagues pass at 68.8%, compared with 49.8% for undisclosed requirements and 73.7% for requirements available in public records (Figure 9b). Within tasks containing both disclosed and undisclosed requirements, the mean difference in node pass rates is 21.3 percentage points. This association is consistent with requirement

acquisition contributing to implementation success, although task difficulty and agent behavior may affect both disclosure and delivery.

Failure stages for linked requirements. Figure 9c attributes 42% of failed nodes with linked requirements to the *ask* stage. This includes 173 nodes whose cards were never raised with their owners and 242 whose cards were raised without obtaining disclosure. Another 56% are assigned to *implement*: the requirement reached the agent, but the tested behavior remains unsatisfied. The remaining 2% are assigned to *discovery*. No failed node in this attributed set is assigned to an unread disclosed reply.

The *implement* category covers failures after requirement receipt, including possible errors in interpretation, implementation and validation. Both categories reflect requirements whose state the agent did not carry forward. In the *ask* category, the owner was usually contacted and the requirement was left unraised or unresolved. In the *implement* category, the requirement reached the agent and the tested behavior was left unsatisfied. The next three subsections examine follow-up, implementation and reporting in turn.

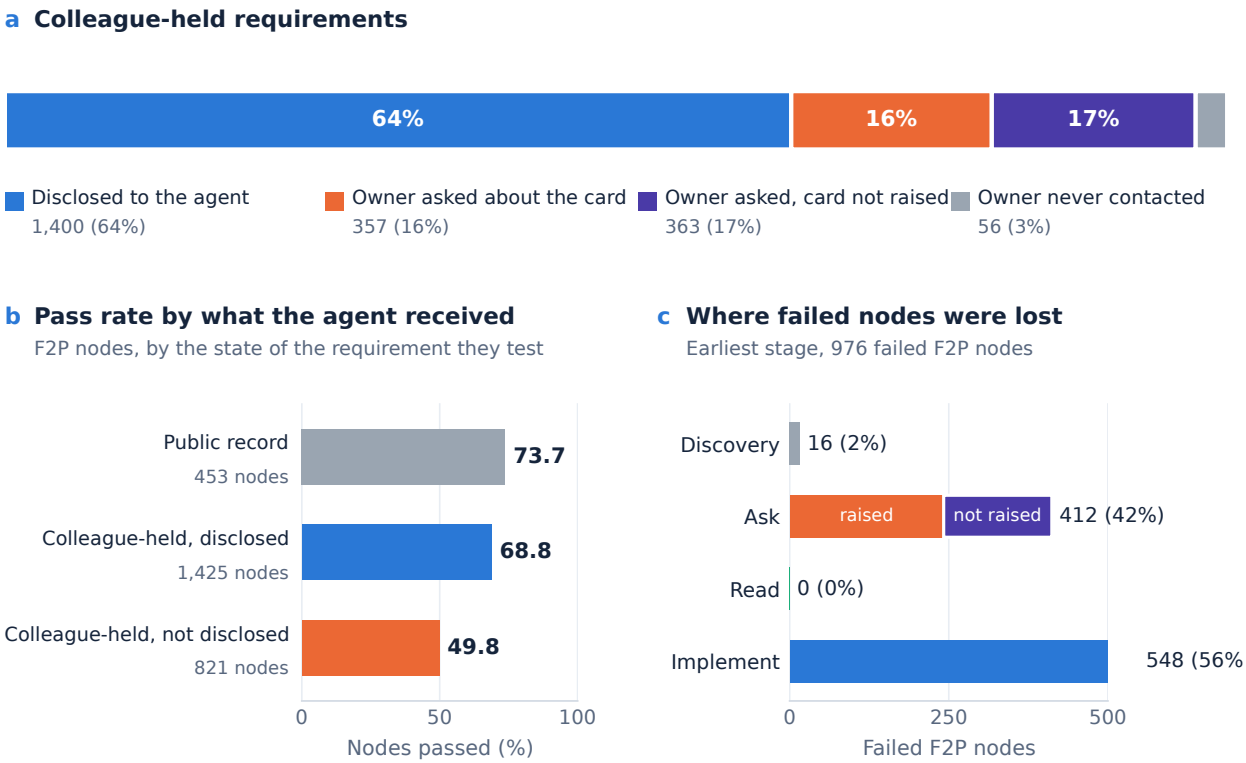


Figure 9 Requirement acquisition and loss in Claude-Opus-5.5. (a) Requirements held by colleagues, grouped by acquisition state. (b) F2P node pass rates against the full release contract, grouped by whether the corresponding requirement reached the agent. (c) Earliest observed loss stage for failed nodes with linked requirements, evaluated against the full release contract. The *implement* category denotes unsatisfied behavior after the requirement reached the agent.

6.3.2 Clarification over time: following up deferred questions

Owners initially defer answers in 450 threads, each associated with an owner and a card, across 96 runs. Claude-Opus-5.5 returns to 297 of these threads, and 205 eventually result in disclosure. The other 153 threads receive no follow-up and produce no subsequent disclosure (Figure 10b). At the initial deferral, the median abandoned thread has 7.1 hours of execution budget remaining. The agent then continues working for a median of 3.4 hours before stopping. These threads remain unresolved despite substantial time spent on other work after the initial question.

Colleague messaging continues throughout execution. Of 2,552 message attempts, 34% occur during the first

fifth of elapsed time and 20% during the final fifth (Figure 10c). The aggregate activity spans the run, while individual questions can remain unanswered after the agent moves on to other tasks.

The PostCSS task (row 014) illustrates this pattern. The agent raises card CSS-227 early, follows up with a concrete example, and receives a second deferral after sharing its own findings. It subsequently declares the streams complete with the question still unresolved. All 16 failed F2P nodes in this run concern undisclosed requirements held by colleagues. The agent conducts useful investigation and initiates follow-up, yet the outstanding answer remains unresolved when it declares completion.

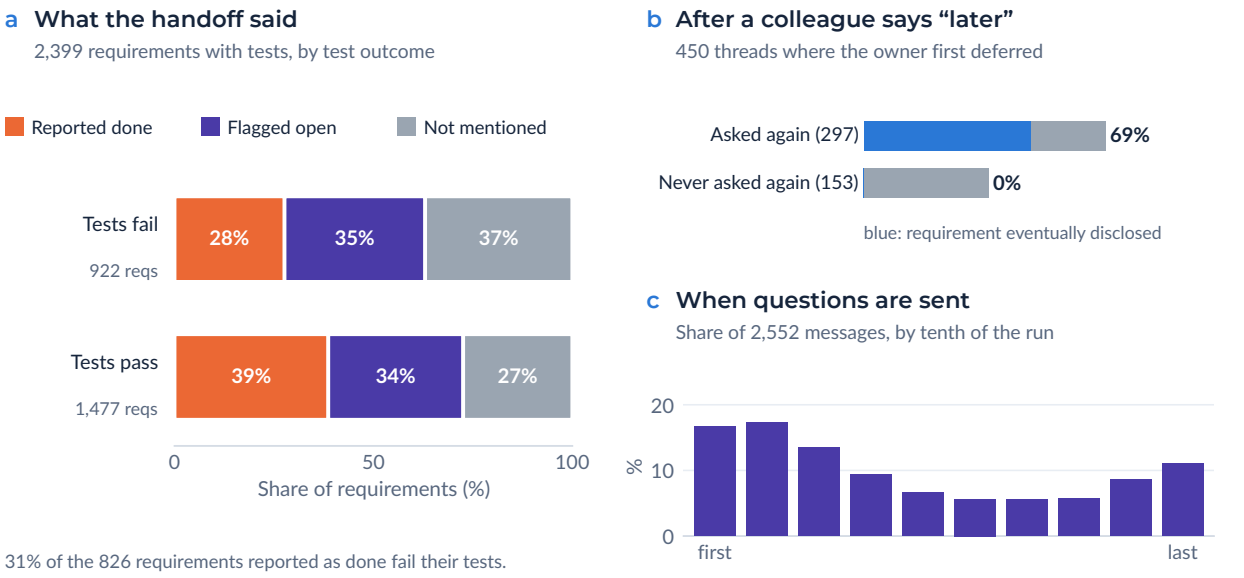


Figure 10 **Follow-through and reporting in Claude-Opus-5.5.** (a) Delivery claims compared with the test outcome of each requirement. (b) Deferred threads grouped by owner and card, subsequent follow-up and eventual disclosure. (c) Timing of colleague messages within a run. Message activity across the run can coexist with individual threads receiving no follow-up.

6.3.3 Implementation and verification of obtained requirements

The carapace-bin task (row 001) illustrates failures that persist after requirement acquisition. Claude-Opus-5.5 obtains all 25 requirements held by colleagues, yet 22/35 F2P nodes fail. The reviewer identifies mismatched usages, flags and candidate descriptions, together with a build failure that remains unchecked. The available requirements have reached the agent, while the implementation and its verification still leave substantial parts of the executable contract unsatisfied.

The exploratory behavior profile provides a broader view of these failures (Figure 11). Higher F2P within the cohort is associated with more effective translation of evidence into code ($\rho = +0.36$), causal debugging ($\rho = +0.30$), verification design ($\rho = +0.30$), and following owner questions through to an answer ($\rho = +0.21$). Raw message count has a weaker positive association with F2P ($\rho = +0.21$), while tool call count has a negative association ($\rho = -0.21$). Elapsed time shows little rank association with F2P (Figure 12a–c). These exploratory associations suggest that the outcome depends more closely on how agents use information and verify changes than on the volume of recorded activity.

A median run contacts 6 colleagues and uses 5.0 hours of the eight-hour budget. The remaining failures occur despite substantial time spent on collaboration and coding. Assignment constraints are also violated in some runs: among 94 briefs that prohibit changes to existing tests, 15 submitted patches change or remove lines in existing test sources. These violations add a separate class of delivery errors to failures in implementing new behaviors.

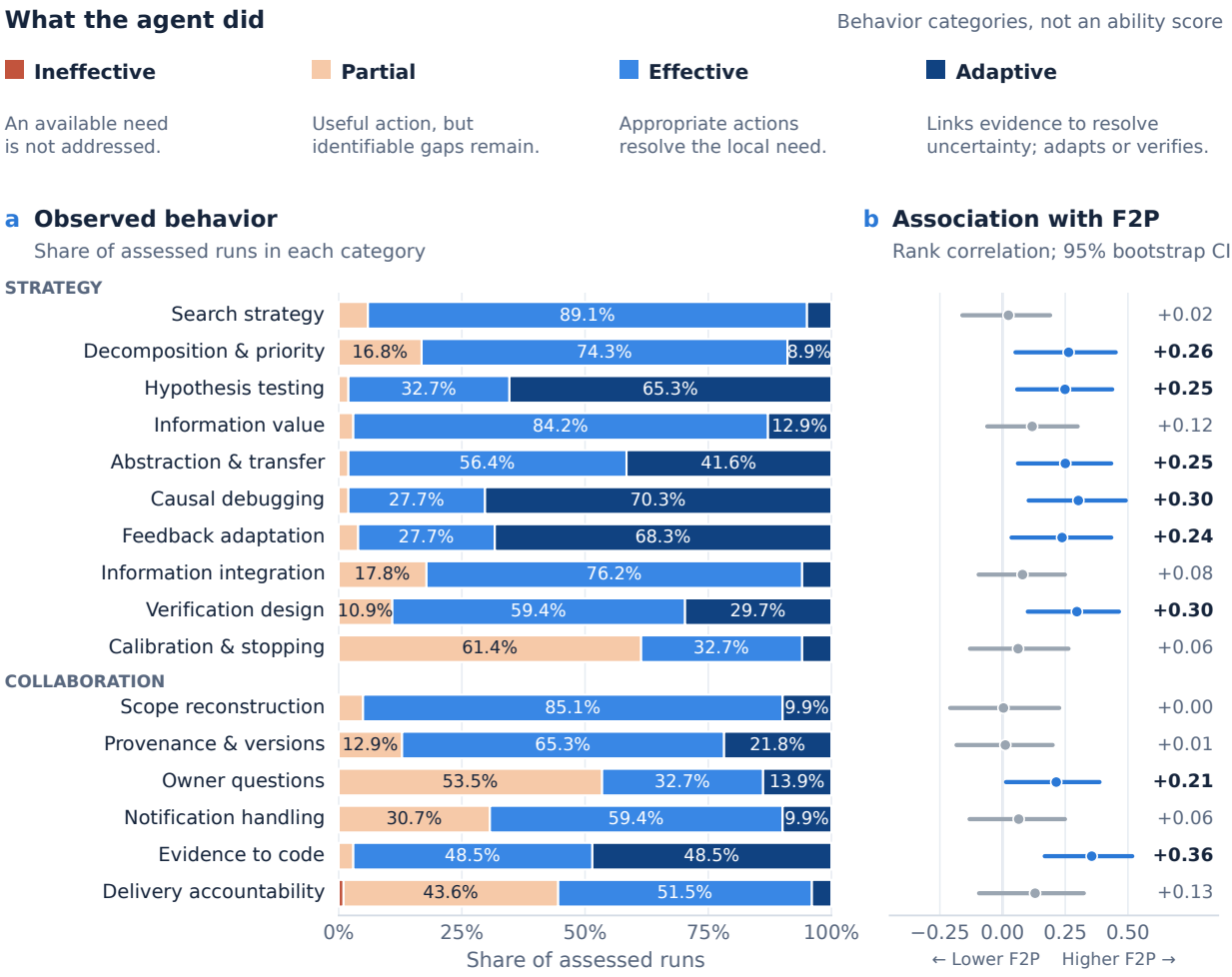


Figure 11 **Claude-Opus-5.5 behavior profile.** (a) Percentages of assessable runs by observed behavior. *Ineffective* actions fail to address an available need; *partial* handling leaves identifiable gaps; *effective* handling resolves the local need; and *adaptive* handling connects evidence across steps to resolve uncertainty and adapt or verify. Each bar is normalized over assessable cases for that dimension. The categories describe local behavior and do not imply success on the whole task. (b) Spearman associations with F2P for each task, with exploratory 95% bootstrap intervals. The categories use evidence defined for each dimension and are not combined into an ability score. Associations do not establish causation.

6.3.4 Handoff: stopping decisions and reported status

Stopping with actionable work remaining. The model reviewer classifies calibration and stopping as *partial* in 61.4% of the release cohort, indicating that useful work has been performed while identifiable gaps remain in resolving or reporting the assignment. It identifies 49 premature stops where an in-scope gap and an available next step remain (Figure 12e). Among the 48 autonomous stops in this group, the median remaining execution budget is 2.8 hours.

The review protocol evaluates ordered trajectories before private grading outcomes are attached. The stopping judgments therefore reflect unresolved work and available next steps visible in the recorded trajectory. In these runs, the agent ends execution while further work on the assignment remains possible.

Delivery reports and remaining work. The claim analysis covers 2,399 requirements across 101 runs (Figure 10a). Of the 922 requirements that fail their tests, final handoffs identify 35% as open, report 28% as done, and omit the rest. Among the 826 requirements reported as done, 255 (31%) fail their tests. Counting

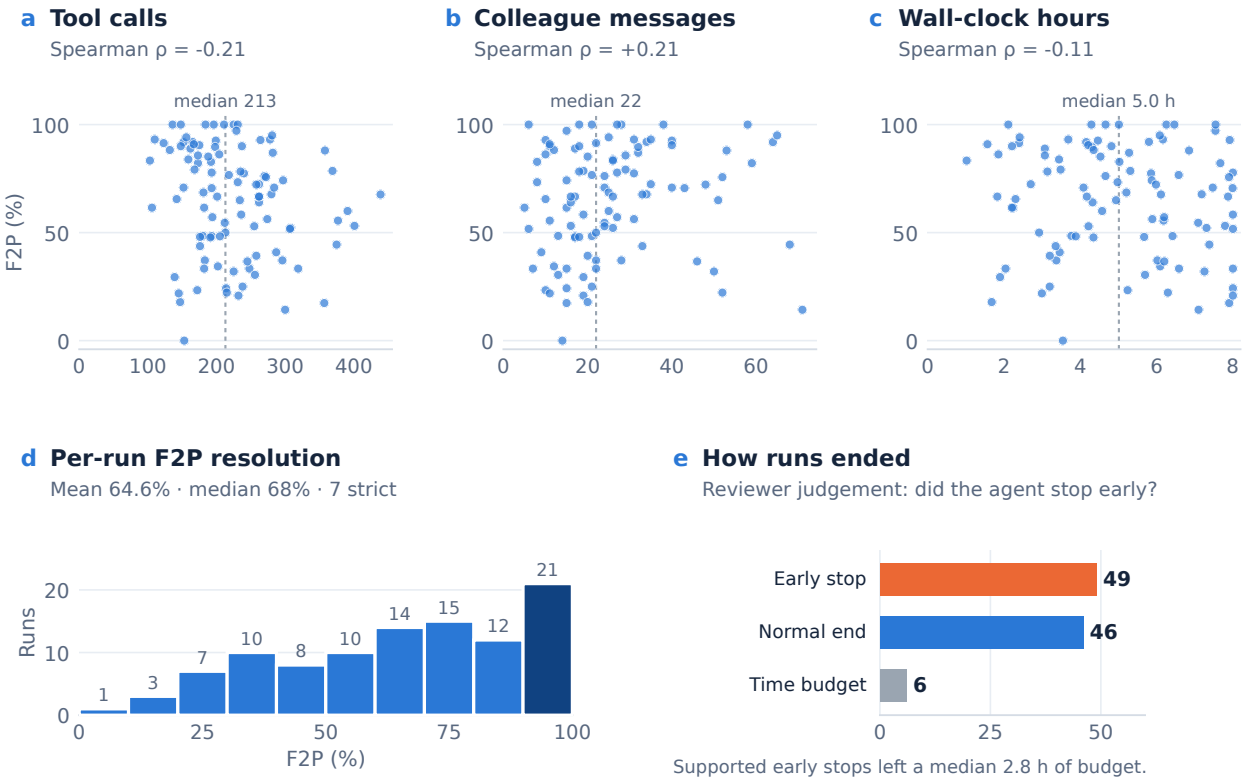


Figure 12 **Effort, outcomes and stopping in Claude-Opus-5.5.** (a–c) Tool calls, colleague messages and wall-clock hours against F2P fractions for each task. (d) Distribution of F2P fractions over the 101-task release cohort. (e) Reviewer judgments of how runs ended, including stops with known, actionable gaps.

all rows of "confirmed requirements" tables as completion claims raises this estimate to 382/1,157 (33%).

The final reports therefore contain both omitted unfinished work and completion claims contradicted by verification. The test comparison measures the accuracy of reported status; evidence of what the agent recognized during execution comes from the trajectory itself.

HotswapAgent (row 021) provides a direct example. The agent declares the assignment complete while listing 8 requirements as pending in its own notes. The completion claim conflicts with the agent's recorded work status. Such discrepancies make it difficult for colleagues to determine which requirements have been completed and which still need attention.

6.4 Tracking Requirements Through an Assignment

The results across models, the Oracle comparison and the Claude-Opus-5.5 trajectories point to the same gap. Strong agents find the accountable colleagues and gather most of the context they need. They lose the state of individual requirements between clarification, implementation, verification and handoff. Claude-Opus-5.5 contacts the owner and leaves the relevant card unraised; it receives a deferral and moves on with 7.1 hours of budget remaining; it obtains a requirement and leaves the tested behavior unsatisfied; it stops with actionable work in view; and it reports as done 31% of requirements that fail their tests. Each of these failures concerns a single requirement whose status the agent stopped tracking.

We use the practice of a strong human engineer as a process reference. This reference is conceptual and does not come from a measured human baseline. Such an engineer keeps an explicit record of open questions, implemented changes, verification results and remaining obligations. An assignment may remain incomplete within a fixed budget, provided the handoff states what has been delivered and what still requires work.

Reliable delegation in the software organization described in the introduction depends on this record. A

supervisor needs to know which dependencies are outstanding, which changes are implemented and verified, and which decisions remain. Weaker models also fall short at the first stage: DeepSeek V4 Pro sends a median of 0 colleague messages per run. For the strongest models, the remaining gap lies in tracking each requirement through to an accurate handoff.

Appendix G reports supplementary Claude-Opus-5.5 collaboration and stratified diagnostics. Appendix I records the diagnostic definitions, claim-classification rules and evidence sources.

7 Conclusion

We introduce SWE-CoWork, a benchmark that evaluates coding agents working within engineering organizations of digital employees. It comprises 101 development assignments across 101 real repositories, with 26 modeled colleagues per organization. Requirements are held by accountable colleagues and project records, and each requirement is linked to a fixed executable test contract. Starting from a brief handoff, the agent establishes requirements, implements and verifies the changes, and reports what it delivered. Across eleven models, the highest F2P score reaches 63.84%, while complete task success reaches at most 7.92%. Supplying complete requirements raises F2P from 39.09% to 55.76% for Seed Evolving and from 62.91% to 78.84% for GPT-6 Astra, with task pass rates unchanged. Claude-Opus-5.5 reaches the accountable owner for 93% of first questions and obtains 64% of the requirements held by colleagues. It then loses track of individual requirements: cards go unraised, deferred questions go unanswered, obtained requirements remain unsatisfied, and 31% of requirements reported as done fail their tests. Tracking each requirement from clarification through implementation, verification and handoff is the central gap these results identify.

Toward teams of autonomous programming agents. We envision engineering organizations in which humans set goals, establish priorities, and make major product and technical decisions, while programming agents coordinate and execute the work. These agents would consult colleagues, resolve dependencies, integrate changes, and report verified results alongside outstanding issues requiring human attention. The results of SWE-CoWork highlight the importance of maintaining responsibility for each requirement throughout collaboration, implementation, verification, and delivery. The current benchmark provides a setting for evaluating an individual agent working within an organization. Future work can extend this setting to teams of autonomous programming agents and measure how much engineering work a human can reliably delegate to them.

References

Alibaba Cloud Model Studio. Models: Qwen3.8-Max and Qwen3.8-Flash, 2026. https://www.alibabacloud.com/help/en/model-studio/models. Official model list. Accessed October 19, 2026.

Anthropic. Introducing Claude Opus 5.5, 2026. https://www.anthropic.com/news/claude-opus-5-5. Official model announcement. Accessed October 19, 2026.

ByteDance Seed. Seed2.1 officially released: Advancing AI productivity, 2026a. https://seed.bytedance.com/en/seed2_1. Official model page. Accessed October 19, 2026.

ByteDance Seed. Doubao-Seed-Evolving model details, 2026b. https://ark.volcengine.com/region:cn-beijing/model/detail?name=doubao-seed-evolving. Volcano Engine Ark model page. Accessed October 19, 2026.

Cursor. CursorBench 4.0, September 2026. https://cursor.com/evals. Version 4.0, released September 10, 2026; accessed September 15, 2026.

Snagnik Das, Siddhant Paliwal, and Janak Sunil. Introducing Real-SWE. Specific Labs, September 2026. https://www.withspecific.com/benchmarks/real-swe. Private enterprise benchmark; accessed September 15, 2026.

Datacurve. DeepSWE v1.1, June 2026. https://deepswe.datacurve.ai/blog/deepswe-v1-1. Updated execution and node-id scoring release; accessed September 15, 2026.

DeepSeek AI. DeepSeek-V4.1-Flash release, September 2026a. https://api-docs.deepseek.com/news/news260910. Official release note, September 10, 2026.

DeepSeek AI. DeepSeek-V4-Pro GA release, August 2026b. https://api-docs.deepseek.com/news/news260813. Official release note, August 13, 2026.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can ai agents solve long-horizon software engineering tasks?, 2025.

Nicholas Edwards and Sebastian Schuster. Ask or assume? uncertainty-aware clarification-seeking in coding agents, 2026.

Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026.

Naman Jain. How we compare model quality in Cursor, March 2026. https://cursor.com/blog/cursorbench. March 11, 2026; accessed September 15, 2026.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Gyuhyeong Kim, Hyojung Gwon, Jeonghyeon Kim, Kyuhong Shim, and Sunjae Lee. RealSWE: A compositional evaluation of coding agents under realistic user requests, August 2026. https://arxiv.org/abs/2608.27831. Version 2, August 31, 2026.

Brendan King and Jeffrey Flanigan. Dialogue SWE-Bench: A benchmark for dialogue-driven coding agents, 2026. https://arxiv.org/abs/2606.13995.

Amy J. Ko, Robert DeLine, and Gina Venolia. Information needs in collocated software development teams. In *29th International Conference on Software Engineering (ICSE)*, pages 344–353, 2007. doi: 10.1109/ICSE.2007.45.

Rishyanth Kondra, Sanket Mhatre, Akshit Kumar, Evan Chu, Bilal Bakht Ahmad, Ayush Nangia, Rajan Agarwal, Arpan Dasgupta, Animesh Sinha, Bhuvanesh Sridharan, Krupa Dave, Brendan Graham, Guanyu Song, Anirudh Rahul, Wei Hern Lim, Abishek Thangamuthu, Ramneet Singh, Danna Liu, Navid Pour, Calvin Chen, and Justus Mattern. FrontierSWE v2. Proximal Blog, 2026. https://www.frontierswe.com/blog/v2. Version 2, 34 tasks, accessed September 15, 2026.

Thomas D. LaToza, Gina Venolia, and Robert DeLine. Maintaining mental models: A study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, pages 492–501, 2006. doi: 10.1145/1134285.1134355.

Haobin Li, Ping Deng, Weizhong Qian, Liang Jiang, Zhenyu Huang, Mouxing Yang, and Xi Peng. Active-SWE: Benchmarking coding agents for proactive bug fixing without issue reports, 2026. https://arxiv.org/abs/2608.04682.

Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. SWE-Lancer: Can frontier LLMs earn $1 million from real-world freelance software engineering?, 2025.

Moonshot AI. Kimi K3 is here: Our most capable model, July 2026. https://forum.moonshot.ai/t/kimi-k3-is-here-our-most-capable-model/480. Official model announcement, July 22, 2026.

OpenAI. Introducing SWE-bench verified, 2024. https://openai.com/index/introducing-swe-bench-verified/.

OpenAI. GPT-6 Astra model, 2026. https://developers.openai.com/api/docs/models/gpt-6-astra. Official model documentation. Accessed October 8, 2026.

Zhongyuan Peng, Dan Huang, Chuyu Zhang, Caijun Xu, Changyi Xiao, Shibo Hong, David Lo, Lin Qiu, Xuezhi Cao, Jiyuan He, and Yixin Cao. ICAE-Bench: Evaluating coding agents as interactive project builders, 2026.

George Pu, Michael S. Lee, Udari Madhushani Sehwag, David J. Lee, Bryan Zhu, Yash Maurya, Mohit Raghavendra, Yuan Xue, and Samuel Marc Denton. LHAW: Controllable underspecification for long-horizon tasks, 2026.

Mohit Raghavendra, Soham Dan, Miguel Romero Calvo, Yannis Yiming He, Johannes Baptist Mols, Gautam Anand, Cole McCollum, Edgar Arakelyan, Vijay Bharadwaj, Andrew Park, Jeff Da, MohammadHossein Rezaei, Bing Liu, Brad Kenstler, and Yunzhong He. SWE Atlas: Benchmarking coding agents beyond issue resolution, 2026. https://arxiv.org/abs/2605.08366.

Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations, 2026.

Quan Shi, Keshav Dhandhania, Karthik Narasimhan, and Victor Barres. $\tau^{\intercal}$-bench: An environment for end-to-end, realistic agent construction, 2026. https://arxiv.org/abs/2609.04611.

Jonathan Sillito, Gail C. Murphy, and Kris De Volder. Asking and answering questions during a programming change task. *IEEE Transactions on Software Engineering*, 34(4):434–451, 2008. doi: 10.1109/TSE.2008.26.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024.

Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. Measuring program comprehension: A large-scale field study with professionals. *IEEE Transactions on Software Engineering*, 44(10): 951–976, 2018. doi: 10.1109/TSE.2017.2734091.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench multimodal: Do ai systems generalize to visual software domains? In *International Conference on Learning Representations (ICLR)*, 2025a.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. SWE-smith: Scaling data for software engineering agents, 2025b. Introduces SWE-bench Multilingual (300 instances, 42 repositories, 9 languages).

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

Z.ai. GLM-5.2 overview, 2026a. https://docs.z.ai/guides/llm/glm-5.2. Official developer documentation. Accessed October 19, 2026.

Z.ai. GLM-5.3 overview, 2026b. https://docs.z.ai/guides/llm/glm-5.3. Official developer documentation. Accessed October 19, 2026.

Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-SWE-bench: A multilingual benchmark for issue resolving, 2025.

Xin Zhou, Chun Yong Chong, Kisub Kim, Yun Peng, Rui Shu, Zihan Wu, Xu Han, Guowen Yuan, Zeyang Zhuang, Jounghoon Kim, Jeongjin Ju, Seongmin Ju, Taein Yoon, and David Lo. A unified issue resolution benchmark for requirement clarification, planning, and code generation for coding agents, 2026.

A Contributions

Leading Authors Xinping Lei

Core Contributors Minghao Liu, Ruijie Zhu, Xiaohui Hu

Contributors Xingqiao Li, Ao Zhang, Wenhao Chai, Han Li, Jingyuan Zhang, Enduo Zhao, Yunfeng Shi, Yiming Liang, Zetao Yang, Xingjian Wang, Yifan Yao, Xinkai Ma, Tianhan Yang, Huan Zhou, Weiran Shi, Shen Yan

Corresponding Authors Ge Zhang, Jiaheng Liu, Weihao Xuan

B Broader Benchmark Lineage

Table 4 compares repository and interactive benchmarks by their research focus, information sources and evaluation design. Table 1 provides a more detailed comparison of recent benchmarks in terms of task goals, initial inputs and information processes. SWE-bench (Jimenez et al., 2024) established patch evaluation in real repositories using F2P/P2P tests. SWE-bench Verified (OpenAI, 2024), SWE-bench Multimodal (Yang et al., 2025a), Multi-SWE-bench (Zan et al., 2025) and SWE-bench Multilingual (Yang et al., 2025b) extend validation quality, modalities and language coverage. SWE-bench Pro (Deng et al., 2025) evaluates larger changes, SWE-Lancer (Miserendino et al., 2025) draws on paid freelance assignments, and Active-SWE (Li et al., 2026) evaluates proactive bug discovery from repository state.

Benchmark	Focus	Information setting	Evaluation / control
SWE-bench (Jimenez et al., 2024)	Issue resolution	Issue and repository	F2P/P2P tests
SWE-bench Pro (Deng et al., 2025)	Complex software changes	Issue and repository	Repository tests
SWE-Lancer (Miserendino et al., 2025)	Paid freelance work	Task description	End-to-end tests for implementation tasks
Active-SWE (Li et al., 2026)	Proactive bug discovery and fixing	Repository without issue reports	Bug discovery and repair evaluation
TheAgentCompany (Xu et al., 2025)	Professional workflows, including SWE	Company services and multiple colleagues	Programmatic checkpoints; some model-based criteria
LHAW (Pu et al., 2026)	Controlled underspecification	Removed goals, constraints, inputs or context	Inherited evaluators and agent trials
Ask or Assume (Edwards and Schuster, 2026)	Clarification under uncertainty	Full issue held by a user simulator	Full, hidden and interactive conditions
Dialogue SWE-Bench (King and Flanigan, 2026)	Coding through dialogue	Persona-grounded user simulator	Task outcomes and dialogue quality
SWE-RPG (Zhou et al., 2026)	Requirement and planning diagnosis	Issue and repository	Executable patches and intermediate reference annotations
ICAE-Bench (Peng et al., 2026)	Interactive project building	Fuzzy requirements and a user simulator	Behavioral tests and multiple diagnostics
τ^{τ}-Bench (Shi et al., 2026)	Agent construction	Business artifacts and a client	Held-out user simulations; serving constraints
SWE-CoWork (ours)	Autonomous task completion	Distributed evidence, multiple owners and late notices	**Same-task oracle control**, F2P/P2P tests and recovery traces

Table 4 Comparison of repository and interactive benchmarks by research focus, information sources and evaluation design.

C Released Repositories

Table 5 lists the 101 upstream repositories represented in the 101 released tasks, grouped by application domain.

Domain	n	Upstream repositories
Web & Frontend	26	ada-url/ada, api-platform/core, collective/icalendar, emberjs/ember.js, feedbin/feedbin, fomantic/fomantic-ui, gioxx/marvelloussuspender, googleapis/google-api-php-client, hyperf/hyperf, ing-bank/lion, ionic-team/stencil, jasontaylordev/cleanarchitecture, jeremydaly/lambda-api, jnunemaker/httparty, jquery/jquery, kimai/kimai, mika-cn/maoxian-web-clipper, nolimits4web/swiper, python/pythondotorg, rubenlagus/telegrambots, saloonphp/saloon, shane32/qrcoder, tanstack/query, urllib3/urllib3, vacanza/holidays, webrecorder/pywb
Dev Tools & Compilers	22	ansible/ansible-lint, carapace-sh/carapace-bin, cargo-generate/cargo-generate, cocoapods/xcodeproj, embarkstudios/cargo-deny, firebase/firebase-tools, git-lfs/git-lfs, hotswapprojects/hotswapagent, isomorphic-git/isomorphic-git, jctools/jctools, jest-community/jest-extended, junit-team/junit-examples, kpdecker/jsdiff, mdx-js/mdx, microsoft/vscode-js-debug, nedbat/coveragepy, pallets/click, postcss/postcss, psf/black, serde-rs/serde, stoplightio/elements, thought-machine/please
Data & Databases	11	alibaba/jetcache, clockworklabs/spacetimedb, dolthub/go-mysql-server, elastic/elasticsearch-rails, friendsofsymfony/foselasticabundle, kotlin-orm/ktorm, mongodb/laravel-mongodb, qishibo/anotherredisdesktopmanager, rubyzip/rubyzip, ruflin/elastica, webtorrent/webtorrent
Games & Graphics	10	caffeinemc/sodium, hubs-foundation/hubs, mezz/justenoughitems, mikke89/rmlui, multitheftauto/mtasa-blue, nasa-ammos/3dtilesrendererjs, sergio0694/computesharp, steamre/steamkit, tencent/puerts, underminer-steam/undertalemodtool
Desktop Applications	7	betterdiscord/betterdiscord, bobotig/python-mss, electronnet/electron.net, ipfs/ipfs-desktop, openutau/openutau, pimutils/khal, sozi-projects/sozi
Cloud & Infrastructure	5	boto/boto3, brightercommand/brighter, kubermatic/kubeone, robusta-dev/robusta, stackstorm/st2
Finance & Payments	5	btcsuite/btcd, ledgerhq/ledger-live, mempool/mempool, stripe/stripe-php, stripe/stripe-ruby
ML & AI Systems	4	huggingface/transformers.js, infiniflow/ragflow, lmcache/lmcache, onnx/onnx
Security & Identity	4	apache/caldera, bubka/2fauth, crytic/slither, pglombardo/passwordpusher
Mobile Applications	3	luoyesiqiu/dpt-shell, mygod/vpnhotspot, react-native-async-storage/async-storage
Observability	2	errbit/errbit, newrelic/newrelic-ruby-agent
Scientific Computing	2	angularsen/unitsnet, google-deepmind/mujoco

Table 5 The 101 released tasks by application domain and upstream repository.

D Evidence Arriving During Implementation

An agent finds a successful browser record and concludes that the consumer path is covered. During implementation, a new message reports a different SDK observation and requests the client version. The agent contacts the owner, provides the request and chunking context, and obtains the expected behavior for the SDK path. The browser record supports the behavior observed in the browser, while the SDK path requires separate evidence. The owner is available for consultation throughout the task, so an agent can also establish this distinction before the new message arrives.

E Evidence Case Examples

Figure 13 illustrates the judgments about document history, evidence provenance and scope discussed in Section 4.3. The examples use workplace records from Stencil, controller-runtime and Click. The Click records also appear in the opening comparison in Figure 1, where the issue identifies the output capture requirement in `DOC-11`.

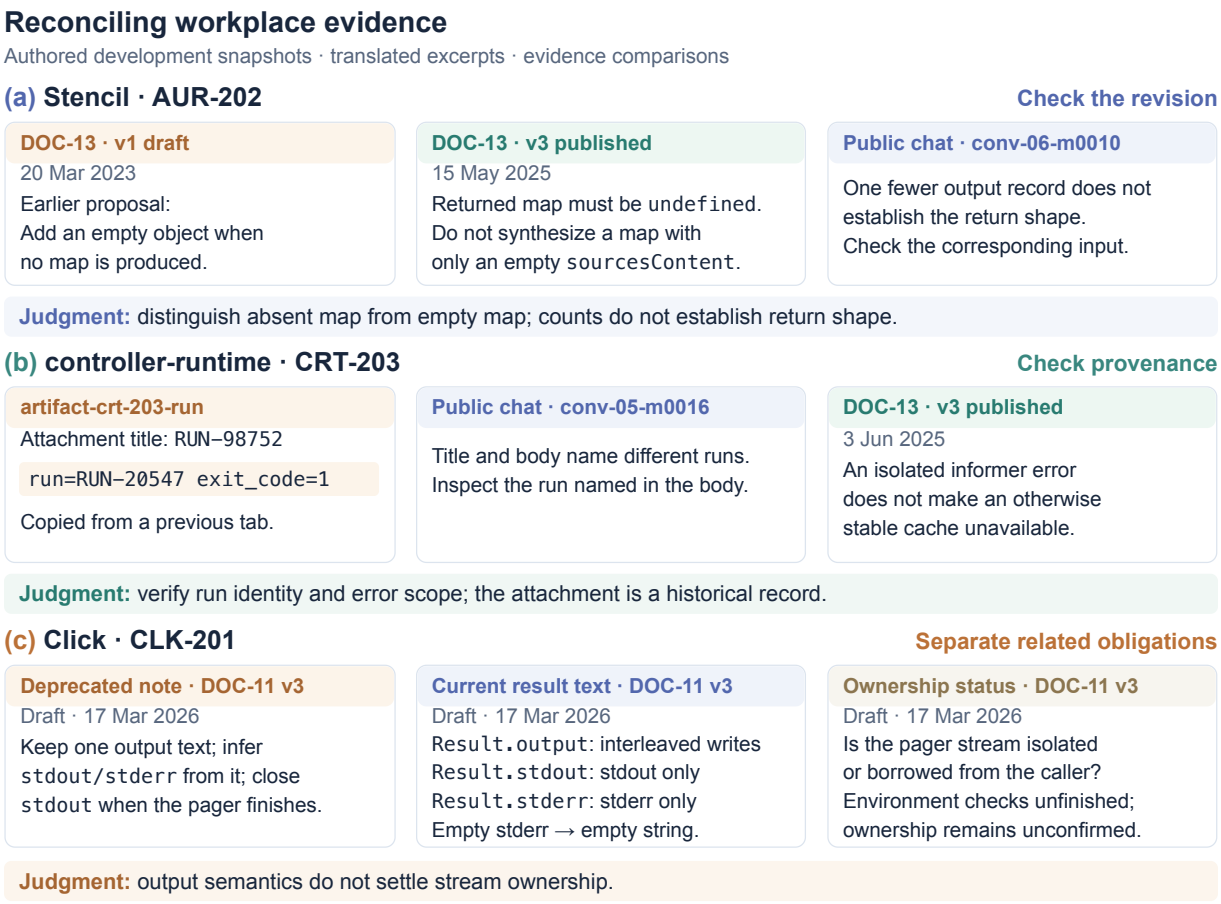


Figure 13 Examples of evidence assessment in workplace records, with source card, document, version and message identifiers. (a) A published correction supersedes an earlier return shape. (b) Run identifiers expose a discrepancy in copied terminal output. (c) Output semantics and an unresolved question about stream ownership.

Stencil: preserving an undefined map field. Card `AUR-202` leads to `DOC-13`, whose published correction supersedes an earlier proposal to fill in an empty map. When `map` is `undefined`, the chunk must still retain its other output information. The linked run excerpt reports 22 versus 21 output records but provides no record shapes. The public reply requests the corresponding input to determine whether the relevant map was absent or empty.

controller-runtime: verifying run identity and error scope. Around `CRT-203`, a public message identifies a mismatch between the attachment title and the run identified in copied terminal output. The linked `DOC-13` also corrects an earlier interpretation that an informer error affected the entire cache. The run identifiers establish which execution produced the observation. Assessing the error's scope requires separate evidence about the state of the otherwise stable cache.

Click: checking output semantics and stream ownership. The handoff on `CLK-201` points to `DOC-11`. Its third version, marked draft, distinguishes interleaved output from output attributed to individual sources

and retains an earlier, deprecated simplification. An incomplete environment check leaves the pager's stream ownership unresolved. Resolving this question requires identifying the component that supplied the stream and whether the stream remains in use after return.

F Per-Task Statistics

Table 6 reports statistics on software changes, workplace records and information design across the released tasks.

Quantity per task	Mean	Median	P25	P75	Max
Code task					
Verifier test nodes	32.3	32	28	35	68
Fail-to-pass test nodes	26.7	27	23	31	67
Pass-to-pass test nodes	5.6	4	3	8	19
Production files in golden patch	97.5	60	37	133	524
Production lines added	4,590.3	2,955	1,687	5,645	26,605
Production lines deleted	1,752.0	1,069	468	2,217	13,232
Workplace					
Colleagues	26.0	26	26	26	26
Board cards	102.3	102	100	104	107
Card comments	62.2	63	58	66	80
Documents	50.5	50	48	52	56
Chat messages	404.3	405	399	410	417
Merge requests	11.8	12	11	13	14
MR review notes	27.8	28	26	30	35
Run artifacts	49.9	50	47	53	64
Information design					
Scored behavioral facts	26.9	28	24	30	32
Contract atoms pinned by tests	28.6	28	18	34	88
Static contract atoms (D_0)	11.1	5	5	14	58
Facts held by an owner (Q)	22.1	23	19	25	28
Progress-gated deliveries (D)	8.0	8	8	8	8

Table 6 Statistics across the 101 released tasks. Production lines and files count source code, excluding tests, documentation, configuration, generated files, fixtures, lockfiles and vendored code.

G Supplementary Claude-Opus-5.5 Diagnostics

The following tables provide additional results from the Claude-Opus-5.5 trajectory analysis in Section 6.3. They report information acquisition behavior and performance across task strata.

	Ask rate	Targeting	Gain/Q	Follow-thr.	Redundancy	Unread
Claude-Opus-5.5 (CoWork)	80.4	93.3	0.64	78.3	13.7	0.2
Spearman ρ with F2P	+0.11	+0.14	+0.04	+0.13	+0.14	+0.18

Table 7 Claude-Opus-5.5 acquisition behavior. The first row reports means across runs; the second reports Spearman correlations between each acquisition measure and task F2P. Ask rate, targeting, gain per question, follow-through, redundancy and unread disclosures are defined in Section 5. All means are percentages except gain per question.

Axis	Stratum	n	F2P (%)	Passed tasks	Disclosed (%)
Verifier nodes	< 28	21	70.1	3	76.5
	28–35	55	62.5	2	61.3
	> 35	25	64.7	2	61.9
Edit size	< 1k production lines	6	67.6	0	75.0
	1k–5k	58	66.0	5	65.4
	> 5k	37	62.0	2	61.7
Language	Python	17	69.3	1	62.2
	TypeScript / JavaScript	32	57.3	1	64.1
	PHP / Ruby	19	68.4	1	68.2
	Go / Rust	11	56.5	0	72.6
	C++ / C# / Java / Kotlin	22	72.3	4	60.2
Domain	Web & frontend	26	75.8	4	72.2
	Dev tools & compilers	22	63.9	2	61.8
	Data & databases	11	70.2	1	67.6
	Games & graphics	10	48.9	0	58.5
	Other domains	32	58.9	0	61.4

Table 8 Stratified Claude-Opus-5.5 diagnostics. F2P is the mean of per-task F2P fractions within each stratum. Table 2 reports a pooled F2P score. Passed tasks counts tasks satisfying both F2P and P2P. Disclosed is the mean proportion of requirements held by colleagues that the agent obtained. Edit size counts production lines added by the reference change.

H Run Behavior and Requirement Acquisition

Table 9 lists the values plotted in Figure 5, together with the median number of colleague messages and wall-clock hours per run. The overall acquisition rate adds the public requirements, which we count as obtained without checking whether the agent read them. The run median computes the colleague rate for each run and takes the median, so it can rank models differently from the pooled rate.

	Median per run			Requirements obtained (%)			
Model	Messages	Tool calls	Hours	Colleague	All	Run median	F2P (%)
GPT-6 Astra	21	178	3.00	67.8	75.3	71.6	62.91
Claude-Opus-5.5	22	213	5.01	64.2	70.7	72.0	63.84
GLM 5.3	23	785	2.46	58.8	66.3	60.9	31.27
Qwen3.8-Max	10	696	8.00	46.9	56.6	48.7	35.70
Seed Evolving	9	503	4.46	44.3	50.6	38.1	39.09
Qwen3.8-Flash	10	902	5.53	40.1	51.0	38.1	30.70
Kimi K3	7	301	1.41	33.1	45.2	27.3	27.08
DeepSeek V4.1 Flash	6	1603	8.00	26.1	39.5	16.0	28.20
Seed 2.1 Pro	8	229	2.50	24.7	36.7	16.3	21.01
GLM 5.2	5	361	2.06	20.4	35.0	12.5	24.40
DeepSeek V4 Pro	0	150	0.90	1.8	23.9	1.5	13.04

Table 9 Run behavior and requirement acquisition. Messages are sent to colleagues through `chat_send`; tool calls include terminal commands and workspace tools; hours are agent wall-clock time, capped at 8. Colleague: share of requirements held only by colleagues that were disclosed. All: disclosed colleague requirements plus public requirements. Run median: median of the per-run colleague rate. Rows are ordered by the colleague rate.

I Claude-Opus-5.5 Trajectories: Measurement Details

Trajectory archive. The analysis covers the 101-task release cohort. Behavioral measurements come from the available Claude-Opus-5.5 CoWork trajectories. Table 2 reports benchmark outcomes evaluated against the complete test contract. Each trajectory record includes its task version, package hash, run ID, requested models, scaffold version, runtime revision and budget. Runs resumed after a provider interruption are treated as single observations.

Linking tests to requirements. For each F2P node represented in the trajectory archive, we map the tested requirements by comparing the per-node requirement files, test source code and upstream change titles with the ORACLE-SPEC for the same package.

Operational loss stages. We use card identifiers, document retrieval records and disclosure events to identify where requirements were missed or lost during information acquisition and use.

Disclosure and visibility. The workplace service logs each requirement disclosed to the agent and links the disclosure to the question that triggered it. A disclosure is considered visible when its reply body appears in a tool result saved in the trajectory. We associate questions with cards using the card identifiers mentioned in the questions. If the agent contacted a requirement's owner but never mentioned its card, we classify the requirement as *owner contacted, card not raised*.

Behavior annotation. The reviewer assesses observable actions and outcomes along ten strategy dimensions and six collaboration dimensions. Each dimension uses four ordered categories: *ineffective* (an identifiable need remains unaddressed), *partial* (useful actions leave identifiable gaps), *effective* (appropriate actions resolve the local need) and *adaptive* (the agent connects evidence across steps to resolve uncertainty and adjust or verify its actions).

The categories are encoded as 0–3 for rank analysis. Each dimension is assessed using its own behavioral criteria. When the trajectory provides insufficient evidence or no opportunity to exhibit a behavior, the dimension is marked *not assessable*. An absent or incomplete dimension review is recorded as missing.

Category percentages are calculated separately for each dimension using assessable runs. Spearman correlations use runs with both an assessable category and an observed per-task F2P score. A trajectory may contribute to one dimension or diagnostic even when another dimension lacks a completed assessment.

For owner-question follow-through, a relevant question that lacks necessary follow-up may be classified as partial. Following the question through to an actionable answer is effective. Resolving consequential ambiguity through linked questions and corroborating evidence is adaptive.

Dimension	Assessed n	I	P	E	A
Search strategy	101	0	6	90	5
Decomposition/prioritization	101	0	17	75	9
Hypothesis testing	101	0	2	33	66
Information value	101	0	3	85	13
Abstraction/transfer	101	0	2	57	42
Causal debugging	101	0	2	28	71
Feedback adaptation	101	0	4	28	69
Information integration	101	0	18	77	6
Verification design	101	0	11	60	30
Calibration/stopping	101	0	62	33	6
Scope reconstruction	101	0	5	86	10
Provenance/version reconciliation	101	0	13	66	22
Owner questions/follow-through	101	0	54	33	14
Notification handling	101	0	31	60	10
Evidence to implementation	101	0	3	49	49
Delivery accountability	101	1	44	52	4

Table 10 Behavior category counts for Claude-Opus-5.5 trajectories in the 101-task release cohort. I: ineffective; P: partial; E: effective; A: adaptive. Each dimension uses its own behavioral criteria. Counts are reported separately by dimension. For cohort presentation, records with incomplete dimension reviews are included in the partial column.

Delivery claims. We examine delivery notes and final completion summaries to determine which requirements the agent reports as completed. Requirements are matched using their card identifiers or, when no identifier appears, their topics. We classify each requirement as *done*, *open* or *not mentioned* based on the surrounding sentence, table row or section.

The open category includes requirements described as pending, unconfirmed, partially completed or dependent on the agent's own assumptions, as well as those listed under an open-items heading. An open designation takes precedence over completion claims elsewhere in the delivery record. Statements referring to a range of cards count as completion claims when they explicitly cover every card in that range.

For the main figures, entries in "confirmed requirements" tables count as completion claims when they also describe an implementation. We additionally report results using the broader interpretation that includes all confirmed entries.

Deferrals and rules. A deferred thread is an owner–card pair for which the owner's first reply postpones the answer. A follow-up is recorded when the agent later messages the same owner about the same card, or when the environment records a qualifying follow-up for that thread.

Remaining budget is calculated from the eight-hour limit and elapsed time since the first saved action. Subsequent working time is measured through the final saved action.

For runs whose brief requires test preservation without a migration exception, we assess compliance by counting lines changed or removed from existing test source files in the submitted patch.